

源码基线提示 (2026-09-15)：本书与 API 清单绑定 `source-baseline.json` 记录的当前工作树快照，基线提交为 `3c7c11c75d15`，并保留未提交源码差异补丁及快照身份。Spec185 的 `PreparedModel/Runtime`、C++ 请求过程和薄 Python 绑定已有当前实现及对应证据；未观测的子解释器、wheel 打包、外部模型与 Spec184 资格仍保持原状态，不能由本书升级。

目录

1 文档范围与源码基线	4
2 系统组成与职责边界	5
3 Core：运行角色、命名与启动	6
4 Core：普通调用与 Targeted 调用	7
5 Core：协作规划与选择事务	8
6 Core：对象传输、连续流与流式调用	9
7 Core：能力、并发、持久状态与观测	10
8 Core：身份验证与请求级保密	11
9 Core：授权版本与权威	12
10 Core：在线授予权限	13
11 Core：在线撤回权限	14
12 Core：版本发现与传播	15
13 Core：授权变化的运行影响	16
14 Repo：架构与对象模型	17
15 Repo：持久化、写入与原子可见性	18
16 Repo：大制品传输与可恢复会话	19
17 Repo：目录、副本、保留与恢复	20

18 DI: 应用入口与当前实现层次	21
19 DI: 输入、模型图、候选与放置	22
20 DI: 计划封装、执行授权与制品	23
21 DI: 原生角色执行与依赖数据流	24
22 DI: 逐 token 生成、KV 与会话延续	25
23 DI: 部署、扩展与管理接口	26
24 UAV: 服务容器、接口与操作界面	27
25 UAV: 飞控适配与操作权限	28
26 UAV: 巡逻任务、补偿与事件协作	29
27 UAV: 视频、遥测流与录像	30
28 UAV: 目标检测与多视角识别	31
29 跨模块协作的完整路径	32
30 构建、配置与部署边界	33
31 实现状态与验证口径	35
32 源码索引: Core 与授权	37
33 源码索引: Repo 与 DI	38
34 源码索引: UAV 与交付	39
35 文档维护与后续目标变更	40
36 API 阅读方法与覆盖边界	41
37 Core API: 容器与角色生命周期	42
38 Core API: 请求发起与 Targeted 调用	43

39 Core API: Provider 注册与作用域回收	45
40 Core API: 延迟规划与协作提交	46
41 Core API: 调用流的事件和终态	48
42 Core API: 命名对象与连续流	50
43 Core API: 在线授权与版本状态	51
44 Core API: 执行租约、观测与并发	53
45 Repo API: 对象与精确 Data packet	55
46 Repo API: 制品上传与提交	56
47 Repo API: 下载、验证与传输状态机	58
48 Repo API: 目录、副本与缓存控制	60
49 DI API: Python 应用入口与原生句柄	61
50 DI API: 模型适配与角色放置扩展	63
51 DI API: 准备、发布后再绑定与封存	65
52 DI API: Provider 注册、授权与执行	68
53 DI API: 生成、KV 与会话延续	70
54 UAV API: 飞控边界与操作权限	76
55 UAV API: 任务、补偿与终端报告	78
56 UAV API: 视频管线、遥测与录像	79
57 UAV API: 检测与多视角识别扩展	80
58 API 扩展与跨模块调用检查	82
59 图解: 四模块关系、对象与关键流程	83

1 文档范围与源码基线

NDNSF、NDNSF-UAV、NDNSF-DI、NDNSF-Repo 完整设计视图

本文覆盖四个模块的职责、主要接口、控制与数据流程、安全、状态、恢复、配置、部署和实现边界。按子系统解释设计，不逐行转录源代码。代码标识、协议字段和文件名保留原名，其余叙述使用中文。

版本 R3；当前设计记录已核对源码行为与明确限制，补充具体接口、状态及例子。目标设计保留独立基线，另列五项 PLANNED 改进。

基线 2026-09-15, Experimental 分支。采样时提交为 3c7c11c75d15；逐文件摘要、工作树差异与本地源码快照见 source-baseline.json。

事实口径 主要流程核对源码；已有接口与已接通流程分别说明。历史测试仅作为检索入口，不冒充本轮结果。

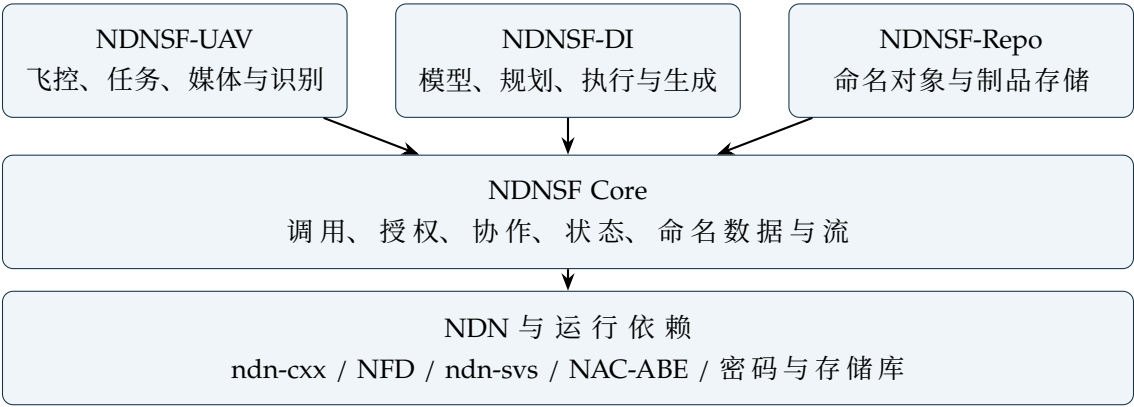
版本维护 本目录完整纳入 Git。两份正文独立保存；修改目标设计不自动修改当前设计或代码。

内容组	覆盖内容
整体与 Core	所有者关系、调用、协作、数据传输、并发与状态。
授权与安全	请求级保密、版本、授予/撤回、状态传播与更新影响。
Repo	对象/制品、持久化、传输、目录、副本与恢复。
DI	应用接口、规划、安全、原生执行、生成/KV、部署与扩展。
UAV	容器、飞控、任务、操作权限、视频、录像及多视角识别。
交付与证据	跨模块流程、运行环境、实现状态、源码索引及维护。

关键现状 当前已验证的原生请求入口是 Runtime::open → User::prepare → PreparedModel → NativeInferenceClient,并由 C++ 过程选择器覆盖 ACK、Selection、Provider、Response、会话和清理。没有 Runtime preparation/request contract 的兼容构造仍明确返回 NATIVE_REQUEST_PIPELINE_NOT_READY；这条限制不能被已验证的 PreparedModel 路径掩盖。

2 系统组成与职责边界

Core 提供通用机制；应用模块拥有领域语义



UAV 可调用 Repo 保存录像，也可调用分析服务；DI 可调用 Repo 发布、复用或获取模型制品。这些是应用间依赖，不意味着 UAV 飞行状态、模型图或 Repo 目录语义进入 Core。[C1, R1, D1, U1]

模块	代码位置与所有权
Core	ndn-service-framework/；通用协议与 C++ runtime。
UAV	NDNSF-UAV-APP/；drone、ground-station、shared 及工具/配置。
DI	NDNSF-DistributedInference/；C++ 角色执行与迁移中的原生请求端、Python 应用和策略层。
Repo	NDNSF-DistributedRepo/；C++ 库/服务与 Python 制品编排。
Python 绑定	pythonWrapper/；Core 绑定与应用辅助层，不等价于全部逻辑均在 C++。

边界规则 Core 可以传递、认证和保存应用定义的 payload，但不解释 waypoint、tensor、model shard、codec 或 object class 的领域含义。

3 Core: 运行角色、命名与启动

身份、服务与进程容器相互关联，但不是同一个对象

ServiceController 维护授权策略和撤权状态，提供签名权限响应及 PolicyStatus，持有 NAC-ABE Authority。

ServiceUser 发布请求、收集 ACK、选择参与者、发送 Selection、处理 Response；维护请求、token、版本与超时状态。

ServiceProvider 注册服务和 ACK/执行处理器；响应候选发现、验证选择、运行任务、发布结果。

ServiceContainer 同一进程组合 Controller/User/Provider 角色和本地服务，管理启动、停止及生命周期 hook。

LocalServiceRegistry 提供可信进程内调用；不是网络调用模式，不增加线上的权限属性或服务选择语义。[C1, C2]

3.1 名字与消息

服务通过 NDN 名称寻址；Request、ACK、Selection、Response 由 Core 的类型和命名 helper 编码。Provider 身份、服务名、requestId 与消息类别共同约束匹配；具体文法以 NDNSFMessages.* 和 utils.* 为准，不由应用拼装新的旁路协议。

通用服务可由多个 Provider 提供；应用需要指向一个物理节点时可使用 Provider 专属服务名，或在已经完成 token bootstrap 后使用 Targeted。名字可达、服务被发现和当前获准执行是不同条件。

3.2 启动与安全准备

加载配置、身份/证书、信任模式，建立 Face 和服务注册，再获取权限和 ABE 材料。构造期异步获取 DKEY 不能阻塞整个应用运行；但 bootstrap-pending 不构成执行授权。Controller 的启动含独立 Face 的公参可达性探测，避免仅以内进程命中宣称 NFD 路径可用。[C2, S3]

配置集中在 ConfigManager、PolicyParser 及应用配置文件；服务容器只负责组合，硬件连接、模型加载和数据库路径由各应用模块决定。

3.3 从配置到可调用服务

以一个进程同时拥有 User 和 Provider 为例：容器身份不是服务名；先配置 Face、证书和 Controller 前缀，再把 /example/echo 注册给 Provider，并保存 scoped registration。容器 start 后事件循环才处理发现与执行。User 取得 requestId 只表示本地请求建立。完整签名、句柄保存与 stop 顺序分别见 AC-02 至 AC-04。

4 Core: 普通调用与 Targeted 调用

先完成发现与选择，再执行；已知 Provider 可使用受保护的短路径

1. User 为新 invocation 分配 requestId、token 和时间预算，发布 Request。
2. 可服务且具有相应权限的 Provider 返回 ACK，携带 Provider token、能力和应用自定义报价。
3. ACK 窗口结束后，User 从候选中选择一个或多个 Provider。策略可为 first-responding、random、all-selected 或自定义。
4. User 发布绑定实际候选与 token 的 Selection；未选中节点不能据 Request 自行开始执行。
5. Provider 验证选择、权限与请求绑定，调用处理器；User 验证返回的签名、密钥和 requestId，再触发结果回调。[C1, C3]



4.1 Targeted 的适用条件

已知 Provider 且已通过正常认证流程取得一次性 token pair 后，后续调用可省略 ACK/Selection。权限、Provider 绑定、请求/响应认证和 token 防重放仍然存在。token 缺失、过期或耗尽时，应走相应 bootstrap/错误处理，不能退化成无认证的直发。[C3]

4.2 失败的归属

Nack、没有可用 ACK、负 ACK、选择失败、执行错误、响应验证失败与超时是不同边界。发布成功不等于执行成功；普通 ACK 也不天然表示独占资源预留。

4.3 应用怎样判断完成

例如请求预算为 3000 ms，应用将 requestId 与界面操作关联，等待 ResponseHandler 或 TimeoutHandler。ACK 阶段不能显示执行完成，也不应启动另一个重复业务请求。普通请求与 Targeted 的并列声明和最小调用见 AC-03；涉及 token 的短路径仍要通过同一服务权限检查。

5 Core: 协作规划与选择事务

不可变 ACK 快照与一次计划提交把应用规划接入同一 invocation

DEFERRED 协作先发起通用请求，再收集 ACK_CLOSED 快照；应用根据快照决定角色、参与者、依赖和密钥作用域。规划完成后调用同一次协作的 commit_plan。PREPLANNED 保留请求前已知计划的兼容入口，两者不能被混成一次动态重开 ACK 窗口。[C3, D2]

5.1 Core 负责的契约

角色标识、精确参与者绑定、依赖名称/作用域、请求与 attempt 关联、选择消息分发及终态处理，由通用协作契约承载。模型切分可行性、UAV 任务分区或 Repo 副本评分由上层判断。

GenericSelectionTxnStore 保存认证后的选择上下文与应用提交 blob；OpaqueSelectionParticipant 提供 prepare、onCommitted、onAborted。事务关联 Provider boot epoch、token、可选 lease、截止时间、payload digest 和存储 key epoch；重放已提交记录也检查当前运行代次。[C4]

5.2 带资源预留的选择

R1 的 DI reservation 是独立协商能力，不能赋予所有普通 ACK 相同语义。启用时 Provider 先完成有界资源预留，再给 positive ACK；选择后提交/准备，未选中或迟到候选收到 NOT_SELECTED。资源最终由本地 lease 到期收回。

选中 source role 本地就绪即可运行；有依赖的角色还要获得其直接前驱的认证数据。不把旧兼容 ReadySet/ExecutionActivate 接口解释为所有当前路径都需要全局启动屏障。[D2]

完成边界 计划提交、Provider 接受、资源准备完成、角色执行完成、终端结果被接受是不同状态；文档和应用状态显示应分别记录。

5.3 一个有依赖的协作计划

设 A 在 P1 执行，B 在 P2 执行且消费 A 的输出。ACK_CLOSED 固定本次候选，应用将 A/B 的参与者、A 到 B 的命名依赖及终端规则写入计划，再以相同 requestId 和闭包摘要提交。Core 接受提交后，P2 仍需验证选择并获得 A 的实际输出。提交失败不能靠提前执行 B 补救。AC-05 将 User 提交与 Provider 选择事务的两个 owner 分开说明。

6 Core：对象传输、连续流与流式调用

按数据语义选择入口，不仅按字节数选择

数据形态	当前接口与用途
完整命名对象	<code>publishLargeNamed</code> 、 <code>fetchLarge</code> 、 <code>fetchLargeExact</code> ；确定名字和版本的模型、 <code>tensor bundle</code> 、 <code>manifest</code> 、录像文件。
连续发布流	<code>StreamPublisher</code> 、 <code>PredictiveStreamSubscriber</code> 、 <code>StreamInfo/StreamChunk</code> ；不断产生的媒体、遥测与状态。
单次调用的事件流	<code>RequestServiceStreaming</code> 、 <code>StreamedResponseWriter</code> ；一个 <code>invocation</code> 内的有界事件、取消和终止，如生成 <code>token</code> 。

6.1 完整对象

请求中可携带小型引用；实际字节按精确 `Data` 名获取并验证。大响应的请求级保密路径按段 `AEAD`，必须具有相应的请求密钥/绑定。`Repo` 增加持久存储和副本语义，不建立另一套无命名大 `payload` 通道。[C5]

6.2 连续流

通用流管理会话身份、`cursor`、名称映射、序列缺口、重排、重复、保留窗口、预测请求与背压。当前 `Stream` 还提供对不透明字节的有界 `FEC` 修复能力；媒体帧类型、编码参数和画面呈现仍归 `UAV`。流恢复必须受期限/保留范围限制，不能无限阻塞后续交付。[C6]

6.3 调用事件流

事件绑定 `service`、`request`、`Provider`、`ControllerVersion` 和 `generation` 等上下文。有界队列、事件保留和重试、`transcript/terminal` 校验、取消与 `deadline` 共同控制生命周期。终态只被接受一次，晚到事件不得复活已经结束的请求。普通调用与 `Targeted` 均须满足各自授权前提。[C7]

选择例子 直播使用连续流；已完成录像使用命名对象；一个推理请求逐 `token` 输出使用调用事件流。三者可以组合，但不能互相冒充。

6.4 对应到代码类型

完整对象的结果是可验证的全部字节；连续流的 `StreamPublisher::push` 输入已经签名的 `Data`，返回 `void`；调用事件流的 `writer.publish` 输入 `EventT`，返回 `bool`，`finish` 则可能抛异常。三种 `API` 的参数和返回值不能混用，分别见 `AC-06`、`AC-07`。

7 Core: 能力、并发、持久状态与观测

公共状态表达事实，上层策略解释业务意义

能力与发现 通用 capability/runtime hint 表达可用性、负载、队列、资源和 drain 状态；应用字段以带 schema 的 payload 附着。可达不代表可以接收新任务。

准入与租约 ProviderAdmissionLeaseTable 和 ExecutionLease 维护有期限的准入/执行状态；request、service、Provider、资源绑定及消费/释放检查防止跨请求复用。

服务生命周期 注册可使用作用域句柄，关闭后按注册代次阻断晚到回调。一个 Provider host 可共享物理资源 lease 状态，避免多个服务各自重复预订同一槽位。

操作状态 通用状态区分排队、执行、等待输入、完成、失败、取消和过期；结果以类型化 metadata 或 DataProductReference 表达。[C2, C8]

7.1 线程与终态

Face 的 I/O 线程、ACK/handler 工作池以及应用后台线程承担不同工作。pending 请求和资源表由相应锁/代次保护；取消和关闭要清理绑定，不能仅删除 GUI 条目。Core 的事件回调、请求 deadline 与应用长生命周期任务仍分开拥有。

7.2 持久化不等于恢复完成

ControllerGenerationStore 管理持久授权版本；RuntimeStatusStore 可选保存签名状态并重新验证；选择事务有自己的 WAL/存储密钥。DI 的请求 journal、UAV 的 mission ledger、Repo 的数据库是上层持久状态，不能用一个文件存在代替全部恢复握手。[C4, S3, S12]

7.3 可观测性

NetworkTelemetry、TimelineTrace 和应用 diagnostics 提供事件与时序信息。应区分排队/网络/验证/装配/计算/呈现时间；成功标记、源码 hash 和原始运行目录需要关联。单项测试或某个统计计数不自动构成端到端性能或安全资格结论。

7.4 一次执行槽位的生命周期

同一物理设备的冲突资源通过 conflictKeys 与 proof 绑定。租约从 Prepared，经 commit 成为 Committed, validateAndActivate 再核对 requester/request/service/plan 进入 Executing, 结束后 release；未选预留可 abort，到期可 cleanup。Provider 重启后的 providerEpoch 变化会使旧代租约不可照用。AC-09 给出完整创建、提交、激活、释放、查询签名与 reasonCode，观测计数按这些状态解释。

8 Core：身份验证与请求级保密

签名验证、服务授权和内容保密解决不同问题

8.1 身份与信任

证书与 Trust Schema 确认 Data 的签名身份及名称是否符合信任规则。Controller 的授权策略决定身份能否使用或提供服务；签名有效本身不等于拥有服务权限。加密证书与签名证书可分开配置，但须满足同一身份及运行要求；NAC-ABE 使用的加密证书需要兼容其密钥封装。[C2, C5]

8.2 默认 V2 请求级保密路径

1. 初始 Request 的加密发现信息面向获授权的 Provider 类别，使用户不必预先知道具体执行者。
2. 选择后，为本次调用准备独立的输入/响应密钥；输入作为 User 签名的精确命名 Data，在输入密钥下加密。
3. SelectionKeyEnvelope 使用选中 Provider 的 RSA-OAEP 封装传递 K_input 与 K_response，绑定接收者证书及请求上下文。
4. 响应使用 AES-GCM；AAD 包含 service、requestId、ControllerVersion、attempt、证书摘要等绑定。分段或事件使用不同 nonce，nonce registry 拒绝复用。
5. 终态/失效路径释放或清零请求秘密；大响应仍需逐段验证及请求绑定，不能退回共享服务响应钥匙。[C5, S7]

8.3 ABE 与其他密钥的范围

ABE 公参和 DKEY 用于属性策略下的授权/保密；请求 AES 密钥用于本次输入/输出；HybridMessageCrypto 仍用于其他服务级、协作或 token 场景。DI artifact grant、UAV 视频内容钥匙和操作权 lease 是各自应用契约，不与这些字段混成一个 epoch。

8.4 证书和公参可达性

安全 bootstrap 需要实际名称路由、Controller/Authority 可达和签名验证。NFD 前缀注册、forwarding hint 或本地缓存只影响数据如何到达，不能取代身份和内容验证；失败要按证书、公参、权限、状态或内容层定位。

8.5 按攻击对象选择检查

签名有效但没有服务属性的 User 仍应被拒绝；具有服务属性但未被选定的 Provider 不能获得本次输入密钥；得到模型文件的节点也不因此具有特定请求的受保护模型 grant。调用者依次核对身份、服务授权、request/selection 绑定与领域 grant，不把四个问题缩成一个 authenticated 标志。

9 Core：授权版本与权威

版本推进、密钥轮换和权限变化是三个不同判断

对象	当前语义
ControllerVersion	两个非零 uint64：generation timestamp 与 epoch；先比较 generation，再比较 epoch。Controller 的同一全局版本用于各服务的 PolicyStatus。[S1, S2]
PolicyStatusData	按服务返回的签名状态：ControllerVersion、有效期、策略摘要、ABE 公参名字和摘要、Controller 证书、相关撤权记录。[S2]
ABE generation	以实际公参 Data name + digest 区分。grant-only 保持不变；withdrawal 轮换。节点据此判断是否清空共享 NAC 缓存。[S3, S6]
PermissionResponse	面向目标身份的权限表，携带 policy epoch。节点替换本地记录并比较权限内容，决定是否需要目标 DKEY 刷新。[S5]
DKEY	当前 KP-ABE 实现按身份持有完整属性策略的解密密钥，并非每次新增一个服务就附加一个独立属性密钥。[S3, S4]

9.1 不能仅从字段推导的结论

advanceAuthorizationEpoch() 同时设置 m_policyEpoch 与 m_requiredKeyEpoch 为新 epoch；这不表示每次调用都进行了 ABE 全局轮换。运行时另行比较公参身份。[S3]

getPolicyStatus() 对每个服务装入相同的 ControllerVersion，再过滤服务级撤权记录；身份级和证书级撤权也被加入状态。当前结构不是独立的“每身份权限修订号”。[S2]

该函数的 policyDigest 来源于配置文件 SHA-256；不能把它当作完整在线 grant/revoke 变更集摘要，也不能仅凭其不变就判断某节点不受影响。[S2]

9.2 权威边界

经认证的业务消息所携带的新版本只能触发状态拉取。安装状态必须验证 Controller 签名、服务、有效期及版本绑定；旧版本被拒绝，同版本但内容冲突也被拒绝。状态与刷新协调器先在副本上验证，再一起安装。[S6, S8, S9]

不能混用的两个结论 “ABE 参数未变”意味着不必全局重发 ABE 密钥；它本身不证明该服务所有在途请求仍满足新的授权状态。

10 Core：在线授予权限

保留 ABE 参数，仅替换目标身份的完整策略

1. 应用或控制入口调用 `ServiceController::grant(identity,service,attribute)`。User 使用 `/PERMISSION/...`，Provider 使用 `/SERVICE/...`。
2. 验证目标及 Controller 状态；若此前有未完成的撤权轮换，先恢复该轮换，失败则拒绝本次 `grant`。证书撤销不可由 `grant` 绕过。
3. 更新目标身份的服务授权和属性集合，移除匹配的服务撤权；显式重新授权可解除该身份的身份级撤权。
4. 替换目标身份的完整有效属性策略，重建查找表，再推进全局授权 epoch。
5. Authority 在该身份下一次请求时生成替换 DKEY。普通 `grant-only` 不调用全局 ABE 参数轮换，也不遍历其他身份重发策略。[S4]

10.1 目标身份如何获得新增权限

权限发现需要应用层续期或显式 `fetchPermissionsFromController()`。仅知道新 `PolicyStatus`，并不自动等价于获取新增权限；User/Provider 对权限响应和状态响应分别处理。

`applyPermissionResponse()` 比较更新前后的权限内容：User 比较服务集合；Provider 比较 Provider/service 记录集合。仅内容确实改变时，标记相关服务等待 DKEY 更新。随后安装状态时，若 ABE 参数未变且存在待刷新标记，则执行 DKEY-only refresh。[S5, S6]

同一 `ControllerVersion` 的多个服务响应合并为一次身份 DKEY 更新；状态先到、权限后到的反向顺序也由同版本安装路径处理。相同代次刷新期间，旧密钥保留到替换密钥安装。[S6, S10]

10.2 其他身份会发生什么

没有权限内容变化且 ABE 公参身份不变时，无关身份不会仅因为更高 epoch 而请求替换 DKEY。不过，Provider 接受已安装状态的版本推进后会重新拉取自身权限表；运行状态失效也仍然可能发生。详见“Core：授权变化的运行影响”一节。[S5–S7]

本流程的限定范围 上述不轮换结论限定于普通 `grant-only`。若 `grant` 入口先恢复了此前失败的撤权轮换，此次调用包含那次轮换的影响，不能归类为纯目标身份更新。

10.3 单项输入与策略物化

给身份 U 新增服务 S 时，调用 `grant(U, S, attribute)`，控制器向已有服务/属性集合插入这一项，再从有效集合重建 ABE 策略。应用无需提交 U 的完整权限表。相同授权已存在可以返回 `false`；证书已撤销等拒绝也可以返回 `false`，所以返回值不能单独区分所有原因。已有待轮换状态时先恢复该状态，`grant-only` 的局部更新描述不能覆盖这个例外。见 AC-08。

11 Core: 在线撤回权限

撤回权限，同时切换后续加密使用的 ABE 代次

1. `revoke(target)` 支持身份、证书或特定身份的服务权限撤回；先验证目标并处理已有 `pending rotation`。
2. 将撤权目标加入集合，持久推进 `ControllerVersion`。持久化失败时恢复本次内存修改。
3. 轮换 `Authority` 的 ABE 主密钥/公共参数代次；公参版本由 `generation` 与 `epoch` 派生。
4. 对已知身份遍历剩余有效属性：先移除旧策略；无剩余权限则不再设置策略，否则设置完整过滤后策略。客户端后续获取对应 DKEY。
5. 后续 `PolicyStatus` 返回当前版本、当前 ABE 公参身份和相关撤权记录；`User/Provider` 拉取、验证并安装后，触发授权与缓存处理。[S3, S8]

11.1 失败处理顺序

授权 `epoch` 已持久化而 ABE 轮换抛错时，不回退到旧版本。撤权保留，设置 `m_abeRotationPending`，调用返回失败。下一次 `revoke`（包括同目标重试）或 `grant` 先执行恢复；恢复会预留更高 `epoch`，再轮换，以免改写已经发布的同版本状态。[S3]

这是 `Controller` 本地失败处理，不表示离线或尚未更新的远端节点已经获知撤权。`Controller` 成功、远端发现状态、客户端重新取钥、受保护操作拒绝旧授权，是不同观察点。

11.2 密码学与运行时边界

新代次下加密的数据不能用旧代次 DKEY 正常解密。已经交付的旧明文、旧密钥或旧代次密文不因此被追溯收回。运行中的操作还依赖本地版本门、请求绑定与失效处理，不能仅用“已换钥”描述全部撤权效果。

在节点安装不同公参身份后，会清理本地共享 `NAC Consumer` 缓存、获取新 DKEY，并清理/刷新 `Producer` 的公参状态。这个共享 ABE 范围大于单个服务的本地 `token` 范围。[S6, S7, S10]

证据边界 本文件描述代码顺序和拒绝路径；没有给出撤权全网生效时延、网络分区下的完成保证或本轮运行测试结论。

12 Core：版本发现与传播

当前通过 NDN 拉取状态；没有独立的 Controller SVS epoch 公告

触发方式	当前行为
权限获取/更新	权限路径触发相应 PolicyStatus 获取;新增权限依赖应用续期或显式 refetch。[S5]
经认证的新版本 hint	协调器检查 hint 是否可信、有效且更新；合并正在进行的拉取，按目标版本确认签名状态。[S9]
临近过期刷新	默认按当前状态到期时间安排，携带当前精确版本；旧精确版本请求本身不能发现新 epoch。[S11]
周期重新检查	NDNSF_POLICY_REVALIDATION_PERIOD_MS 大于 0 时使用固定周期，并发出不指定版本的 fresh 状态请求。默认值 0，关闭此周期模式。[S11]
可选状态恢复	恢复持久状态先重新验证，再向 Controller 做在线确认；这不是对离线最新状态的证明。[S12]

12.1 Interest 与 Data 契约

不指定版本的请求前缀为

<controller>/NDNSF/POLICY-STATUS/<service>

客户端设置 CanBePrefix=true 和 MustBeFresh=true；Controller 用当前精确版本名返回签名 Data, FreshnessPeriod=0。指定版本时客户端关闭前缀匹配；Controller 若已不提供该版本则不回答。客户端验证 Data 名字、内部版本和期望版本的一致性。[S8]

12.2 SVS 在当前设计中的位置

ServiceController 的当前实现未建立 epoch 专用 SVS publisher/subscriber。框架其他业务使用 SVS, 不代表授权更新已经具备相同公告路径。本文件及首版目标设计均不把“SVS 通知后拉取状态”方案写成已实现能力。

时效边界 仅推进 Controller epoch 不会主动唤醒所有节点。周期模式、显式拉取和消息 hint 的触发条件决定何时发现变化；本轮没有测量发现或完成时间，也不承诺所有节点立即同时切换。

13 Core：授权变化的运行影响

本地已实现部分区分，但尚非“无关节点完全无感”

设 Provider A 获得服务 S 的新权限，Provider B 的权限内容没有变化，ABE 参数仍为 P，Controller 版本由 V 变为 V+1。以下描述 B 或 User **实际接受服务 S 的新状态**之后的代码分支，而非自动广播后的全网结果。

状态或动作	当前结果
B 的 ABE DKEY	比较权限内容无变化，且公参身份未变时，不执行 DKEY-only refresh。Authority 也没有替换 B 的策略。[S4–S6]
B 的权限表	Provider 从已有状态推进版本时，触发自身权限 refetch；LocalMock 跳过这一网络动作。[S7]
服务 S 的本地状态	新版本安装调用 invalidateControllerScopedCaches，并未先判断变更是否仅属于另一个身份。[S6]
User 的在途调用	同服务 stream 被拒绝；普通调用保留 pending/deadline，但清除请求秘密、绑定、token 等，让终态/超时路径继续负责结束。[S7]
Provider 的在途状态	清理匹配服务的 pending 请求、token、stream 绑定和相关发布状态；不是按 grant 的目标身份过滤。[S7]
其他服务 T	本次服务 S 的失效按服务匹配。但各服务状态携带同一全局 ControllerVersion；若稍后接受 T 的更新状态，T 也会进入自己的失效路径。[S2, S6, S7]

13.1 当前设计的准确结论

可以说：普通 grant-only 只替换目标身份的 ABE 策略；无关身份不因 epoch 变化而重新取 DKEY。

不能说：普通 grant-only 只影响目标 Provider，其他节点无须拉取或更新任何状态、所有在途调用不受影响。这超过当前运行状态失效逻辑所支持的结论。

13.2 当前行为与已接受目标

本页记录全局 ControllerVersion 的基线行为。目标 TG-01 已接受细化影响范围的方向，字段线格式、通知承载及轮换粒度仍待后续 Spec 定稿；NDN-SVS 是可选通知方案，尚未据此建立当前实现的广播保证。目标还须规定版本兼容、离线补齐和在途请求处置。

14 Repo：架构与对象模型

命名对象存储与领域无关；C++ 库和 Python 编排各有边界

RepoCore 不依赖 NDNSF 网络调用的进程内存存储门面；提供 put/get/list/remove、manifest、Data packet、容量和目录操作。

RepoNode 将 RepoCore 挂接到 ServiceProvider 或 LocalServiceRegistry，注册存储操作服务，管理请求响应及 operation 状态。

RepoClient 提供直接、进程内和网络 helper；Python 的 py_repoclient 进一步提供副本、目录和制品会话编排。

存储后端 普通对象/packet 的持久后端与大制品的 PayloadStore/MetadataStore 接口分别承载不同物化方式。[R1–R4]

数据对象	关键身份与含义
RepoObjectManifest	objectName、objectType、SHA-256、size、segmentCount、replicationFactor、replicaNodes、policyEpoch 及生命周期 metadata。
已签名 Data packet	按原始 NDN 名称与 wire bytes 保存；保持发布者身份，不能因进入 Repo 就改成 Repo 内容签名。
ArtifactReference	不可变制品身份：逻辑名、内容摘要、大小、root manifest 名称、发布者与策略 epoch；不是只有路径。
目录条目	逻辑对象、manifest 摘要、来源 Repo、状态、目录版本及副本位置；不等于全部段字节。

14.1 控制面与数据面

默认服务前缀为 /NDNSF/DistributedRepo。CAPABILITY、INSERT、MANIFEST、INVENTORY、STATUS、CACHE_STATUS、CATALOG 系列和 DELETE 等属于控制操作；具体 C++/Python 可用操作以各自注册表为准，不能假定两个入口完全对等。数据面仍通过原始精确名称获取分段 Data，forwarding hint 可帮助定位副本。[R2, R7]

通用性 Repo 不解释模型层、视频帧和张量。payload 是否加密、如何解码和何时可以用于业务，由发布者、消费者及应用权限机制决定。

14.2 对象引用的阅读顺序

先看 objectName 定位逻辑对象，再用 manifest 中的摘要、大小、segmentCount 和有序 packet 名验证内容，最后核对 replica metadata 的实际可读性。模型文件和录像 packet 可以共用 Repo 存储，但前者的模型语义由 DI 解释，后者的签名原始 wire 由 UAV 验证。AC-10 给出两份 packet 的保存/获取例子。

15 Repo：持久化、写入与原子可见性

先提交权威存储，再更新缓存；收到字节不等于发布完成

15.1 普通对象与 packet 存储

部署中的普通 Repo-node 对象路径使用 SQLite 权威存储，叠加按字节预算限制的 LRU 热点缓存。写入先提交数据库，再纳入缓存；读取 miss 回到数据库；删除/覆盖先完成权威事务。超过缓存预算的条目旁路缓存；缓存和计数随进程消失，持久对象不会因缓存清空而被视为丢失。in-memory backend 是测试替身，不应当成受支持的持久部署模式。[R3]

15.2 大制品后端

PayloadStore 管理分范围写入、verified ranges、flush/finalize/abort；MetadataStore 管理生命周期事件。RepositoryStoreFacade 结合后端所有权 lease，防止多个 owner 无约束地修改同一存储目录。FilesystemArtifactStore 和本地 Python artifact backend 支持文件/staging 物化，与普通 SQLite 对象 API 不能混称为一个实现。[R4]

15.3 网络写入流程

1. 发布者确定精确制品身份和 manifest，公开供选中 Repo 拉取的数据源。
2. 通过 NDNSF 协作发现容量/能力，固定副本选择和 operationId。
3. 各 Repo 获取、校验并持久化数据，产生绑定 operation、artifact 和 Repo 身份的提交 receipt。
4. 客户端验证收到的 receipt 与已提交的选择集合及内容身份一致，满足要求后才返回发布完成。

分段收到、哈希验证、持久提交和对外可见分别记录。副本不足或 receipt 不匹配不能用“部分上传成功”包装成完整发布。[R5, R6]

范围区别 普通存储 helper 的重试/分段适配器与 artifact-manifest-v2 的会话生命周期不是同一层；不能把某一条本地路径通过当作所有网络后端都通过。

15.4 成功信号属于哪个阶段

普通 put 的本地返回、制品 transfer 完成、commit 产生已验证 receipt、目录观察到对象是不同事件。断点状态只能支持继续接收，不能让未验证片段直接对外宣称完整文件。文档中的原子可见性指对存储入口的提交边界，不向所有 Python 编排、缓存和跨 Repo 目录扩展为一个分布式原子事务。

16 Repo: 大制品传输与可恢复会话

公开 API 隐藏 packet 编排, 保留精确身份和提交检查

接口	语义
<code>publish_file / publish_file_async</code>	校验源文件身份, 发布到要求的副本数, 返回不可变引用及副本结果。
<code>fetch_file / fetch_file_async</code>	按引用获取并校验文件, 完成后物化到目标位置。
<code>begin_upload → upload_file → commit</code>	显式上传会话, 区分传输与提交。
<code>begin_fetch → transfer → commit</code>	显式获取会话, 区分下载与最终可见文件。
<code>abort(preserve_progress=True)</code>	保留已验证 staging, 允许在相同身份与恢复条件下续传。

16.1 清单、分段与背压

artifact-manifest-v2 将 root manifest、清单页和实际 payload 分开, 避免把海量段列表塞进控制消息。AdaptiveArtifactTransfer 跟踪 in-flight、重传、重复和 verification backlog; 收到段先离开 Interest 窗口, 但在验证/持久化完成前继续施加背压。恢复会话绑定 artifact、root digest、chunk/packet 参数和 lease; 不能把相同文件名当作可续传证明。[R4–R6]

16.2 当前网络入口的限制

CollaborationArtifactApiBackend 承载网络制品发布; 当前 `_NetworkPublishDriver.transfer()` 明确拒绝 TARGETED 整制品发布, 要求 Collaboration。API 中存在 Targeted 控制选项, 不表示每个具体 backend 的所有操作都支持它。[R6]

默认路径按能力与资源边界选择副本, 并验证选中集合对应的持久 receipt。本地单副本 backend 用于本地运行/开发, 不代表已经经历真实 NDN 多副本传输。

恢复原则 只复用经过验证的精确进度; 过期 lease、不同 digest、不同分段参数或错误 operation 绑定应由会话检查拒绝, 不能静默接着写入另一个对象。

16.3 断点下载的操作身份

同一 artifact 下载到不同目标路径可能形成不同 operation。`fetch_file` 默认 `resume=True`、`verify=True`、`replace=False`; 先 `begin_fetch`, 再 `transfer`, 最后 `commit`。异常时依据 `resume` 决定是否保留进度。AC-11、AC-12 展开 descriptor/result、receipt、副本计数、kwargs 和底层 receive/markVerified 的区别, 避免把进度条完成当成文件提交。

17 Repo：目录、副本、保留与恢复

目录负责定位和生命周期；对象字节仍由数据面获取

每个 Repo 有本地目录，Python 控制编排还提供目录交换、查询和修复工具。系统没有把单一集中 NameNode 设为所有对象读取的必要前提；各入口实际是否启用 peer 同步，取决于所选服务和配置，而不是仅因为存在 catalog 类型就自动发生。[R1, R2, R7]

17.1 目录接口

CATALOG_STATUS 返回 Repo 模式、目录 epoch、对象数量和副本接收能力。

SNAPSHOT / DELTA 提供全量恢复或指定 epoch 之后的变化；目录 sequence 不等于 Controller 授权 epoch。

C++ LOOKUP catalogLookup 仅接受精确 objectName 并返回一个条目，没有组合过滤参数。

Python QUERY metadata 过滤和目录编排属于 Python 路径；不能归入 C++ LOOKUP 的能力。

17.2 副本选择与确认

候选 ACK 中的 freeBytes、mode、负载等用于 placement；选中不代表已经保存。策略需满足复制数量/容量约束，真正成功由验证后的写入 receipt 确认。已知副本位置可优化请求，但仍保留名称、签名、manifest 和内容摘要检查。

17.3 删除、TTL 与修复

删除以 tombstone/目录状态表达，避免旧 AVAILABLE 记录无条件恢复对象。对象可带 ttlMs 与 repairAllowed；过期或不允许修复的对象不应作为新副本修复来源。恢复机制需区分 metadata 恢复、字节验证和实际 replica 完成，不能仅依据目录列表计副本数。[R7]

17.4 应用集成

UAV 的内嵌 Repo 为加密录像和原始 packet 留存提供存储；DI 使用公开 artifact API 传输模型制品。独立 Repo-node、应用内 RepoCore 和 Python 网络制品后端是不同部署形态；文档中的能力应绑定具体入口。

17.5 一次目录读不等于副本验收

catalogLookup 在目录锁外读取 store manifest，再在锁内附加本 Repo 的 epoch，因此不是存储与目录的全局一致快照。missing/store 错误沿路径传播，不是统一返回空集合。P1 的 epoch 不能作为 P2 的 delta 游标；目录条目指向 P2 也仍需对实际字节取数验证。精确查询能力和 Python 编排边界见 AC-13。

18 DI: 应用入口与当前实现层次

原生 PreparedModel 是请求入口；Python 只做薄适配和编排

ndnsf_distributed_inference.api 导出 Runtime、User、PreparedModel、句柄和 Provider 的 pybind 对象；api/_async.py 只转换 completion/reader 到 asyncio。旧 app_sdk/APPClient 编排仍保留为兼容 caller，不能把兼容入口的状态外推为 PreparedModel 已覆盖的全部能力。[D1]

18.1 两类应用请求

模型意图请求 以 ModelRef、task、input 和 timeout 提交；AutomaticPlanningCoordinator 在 ACK 窗口关闭后产生图与分割候选、验证 Provider offer、决定计划并提交。

显式预规划请求 使用 RequestableDeployment 或既定 plan；保留兼容 API，不应与无需应用先指定 Provider/role 的延迟规划语义混用。

18.2 C++ 已有的能力

Runtime/User/PreparedModel 将已验证模型包、输入投影和请求句柄绑定到 NativeInferenceClient；NativeExecutionPlan、NativeProviderSession/Runtime、ProviderRoleWorker、模型 runner、依赖传输、生成协调器及 Provider host 负责后续阶段。原生规划、grant、ONNX、tokenizer、会话和请求生命周期由 Spec185 的 C++ 证据逐项覆盖。[D3–D6]

18.3 当前路径与限制

PreparedModel 的 request/run 会把准备包、输入、选项、绝对 deadline 和策略交给 NativeInferenceClient，过程资格已由同一候选的 C++ selector 验证。直接构造没有 preparation/request contract 的 NativeInferenceClient 仍保留 NATIVE_REQUEST_PIPELINE_NOT_READY，用于明确区分兼容/组件构造与生产 prepared 路径。Python subinterpreter、wheel packaging 和 Spec184 外部模型/实验资格仍未观测。[D7]

目标与基线 目标 TG-02 继续要求同一原生实现配可选 Python 外壳；当前 PreparedModel 路径已经是该方向的已实现部分，兼容入口和未观测外部资格仍单独记录。

18.4 如何选择入口并解释失败

需要现在的应用编排时定位 app_sdk/client.py 的 APPClient.request；需要检查原生迁移则定位 NativeInferenceClient.cpp 的 dispatchOperation。模型/任务/输入是业务值，native handle 是异步操作句柄，两者不能替代。AC-14 说明输入形式、默认预算、局部等待和取消；全局实现状态集中见第 31 章，目标所有权迁移见 TG-02。

19 DI: 输入、模型图、候选与放置

先冻结真实观测，再做模型相关的规划

1. 应用输入经过 adapter 预处理，绑定模型内容/语义摘要与输入类型；大输入使用命名引用。
2. 以 DEFERRED 协作获取不可变 ACK_CLOSED，绑定 request、attempt 和总 deadline。
3. 由固定 adapter 描述模型并检查图，枚举有界、图有效的 split candidates。
4. 从真实 ACK payload 构造 ProviderOfferV3；检查身份、offer 签名、Core 认证来源、模型/图绑定与新鲜度。
5. 放置策略评估候选的完整 role/rank、backend/device、内存、队列和驻留证据；独立验证策略输出，不把策略返回值直接作为执行授权。[D2, D3]

19.1 驻留证据与准备

loaded、assembled、canonical 与 cold 表示不同准备成本。has_model 不能证明 exact residency；加载证据还需匹配进程/boot、runtime、设备、artifact、graph、role 和有效期。具有所需服务能力但尚未加载的 Provider，可以在契约允许时接受带准备的执行；不能通过伪造“已驻留”来获得资格。

19.2 策略扩展

ModelFamilyAdapter 拥有图/切分/物化语义；ModelPlacementStrategy 选择 Provider 与设备。Pre-SplitFirst 系列优先复用合法、精确匹配的候选与驻留证据，再考虑准备成本。策略受候选数量、期限和验证器限制，不能扩大自己的权限或绕过 Controller。

19.3 当前原生迁移

NativeRequestPreparation、NativeOfferAdmission、NativeV3Placement 已有对应组件和检查；当前真实数据来源端口及 requester 主链仍需接线。原生组件的定向测试不能替代模型意图经过网络 ACK 到终端结果的整体验收。[D3, D7]

19.4 用两段模型读懂数据结构

线性图分为 A、B 两段，A 输出必须匹配 B 输入，这一分解形成 candidate。认证报价分别给出 P1/P2 的角色、设备与资源，placement 才提出 A→P1、B→P2 的 proposal；validator 再校验它。模型描述、图快照、candidate、offer、proposal 依次收窄自由度，不是同一份 JSON 的五个别名。AC-15 给出精确复用与需准备路径的例子，AC-16 解释发布后如何重绑身份。

20 DI：计划封装、执行授权与制品

计划完整性、权限与制品保密分别绑定

计划包含模型/图身份、角色与 rank、Provider assignment、依赖边、设备和执行约束。V3 的 proposal、plan core、Provider grant view 与 selection projection 分开表示不同可见范围；规范编码和摘要用于验证绑定，不能以临时 JSON 字符串或目录顺序代替身份。[D2, D4]

20.1 从决策到选择

1. 验证完整候选与 assignment，确定需要的可信物化和制品。
2. 通过负责该模型的 adapter/assembler 生成合法切片或装配产物。
3. 使用公开 Repo API 发布/确保制品可获取，保留内容与 manifest 身份。
4. 封装不可变计划与授权绑定，向选中 Provider 提交其最小必要投影。
5. Provider 验证 selection、计划、角色、lease、证书/代次与制品绑定后再进入准备/执行。

20.2 原生安全组件

NativeGrantClient/Verifier、NativePlanSealer、NativeProtectedArtifactStore、NativeProtected-Provider 和 ProtectedRuntime 分担 grant、验证、受保护制品和明文生命周期。ProtectedRuntime 的 lease/zeroization 组件用于显式受保护路径；不能因此推导所有普通模型加载或默认 Python 路径都已启用同等的模型制品加密。[D4]

20.3 三个不同的安全问题

Core 的请求保密保护调用输入/输出；DI 的模型/计划身份保证执行的是被指定的内容；受保护 artifact grant 控制特定制品的解密与运行时明文持有。三者必须分别说明，不能用“使用了 ABE”概括所有模型权限。

当前限制 Spec182 的封装、grant 和准备 owner 虽已有实现片段，真实请求端接线与全部验收尚未闭合；接口和类型存在不等于默认网络路径已使用它。

20.4 五种摘要不互换

ACK 闭包摘要绑定观察到的候选；模型/图摘要绑定被检查内容；artifact root 绑定发布的业务制品；plan core 摘要绑定执行内容；grant/projection 绑定授权视图和具体参与者。发布切片后必须以真实 root/recipe 再绑定角色，再封存计划，不能改下载 URL 却保留旧驻留证明。AC-16 按 prepare、inspect、ensure、bind、seal、grant、project、encode 说明前后数据。

21 DI: 原生角色执行与依赖数据流

角色执行在本地就绪后推进，依赖边通过认证命名对象连接

组件	主要职责
NativeExecutionPlan	角色、依赖边、张量与执行身份；NativeProviderAssignment 指定实际 owner。
NativeProviderSession	将计划、assignment、DependencyIo 和 runner factory 组合成会话。
NativeProviderRuntime	维护 runner 注册与 worker，安排角色执行。
ProviderRoleWorker	等待/预取直接输入、运行角色、按依赖边发布输出；管理局部状态和缓存。
NativeProviderHandler	将 Core CollaborationContext 接到原生 session,验证执行绑定、readiness 与授权。

21.1 输入与输出

TensorBundleCodec 定义张量 bundle 编解码；模型 adapter/runner 验证 dtype、shape、名字、图输入/输出及必要的边界转换。DependencyIo 把数据流边映射为精确命名对象，在 producer/consumer/request/attempt/plan 等绑定下获取和发布。同机角色可使用本地 I/O；网络角色通过 NDNSF 的数据传输接口协作。[D5]

21.2 调度与资源

DependencyWaitScheduler/AsyncDataflowRuntime 负责在依赖可用时推进，避免把等待网络的任务无限占住执行线程。CollectiveRuntime、ProviderGroupCoordinator 等组件表达 group/rank 协调；这些类型和执行路径的存在不能自动作为多 GPU、扩展性或吞吐测量结果。

21.3 模型运行器

OnnxRuntimeModelRunner 以及 ONNX/Qwen/YOLO adapter 处理各自的模型格式和运行契约。设备、CUDA/CPU execution provider、runner 身份与 runtime ABI 必须按实际配置核对；不能把允许 GPU 的配置当成实际使用 GPU。输出发布和 terminal role 接受仍由当前执行绑定约束。[D5]

21.4 A 到 B 的实际执行

Session 组合 plan、assignment、DependencyIo、runnerFactory 与 worker 数。A 的初始输入可由 initialInputs 提供；worker 运行后通过已绑定的依赖 scope 发布 TensorBundle，B 在对应前驱输入就绪后运行。executeRoleAsync 返回 future，异常需由调用者取回并收尾；发布一个中间张量不等于终端结果被 User 接受。session、I/O 和 runner 生命周期须覆盖 future 的工作期，见 AC-17。

22 DI: 逐 token 生成、KV 与会话延续

不可变模型缓存与带身份的推理状态缓存分开管理

22.1 生成循环

NativeEpochCoordinator 组织每轮生成：读取输入/反馈、校验 lineage、执行角色、抽取 logits、按生成配置采样 token、发布 token/终止事件，并处理取消与截止时间。QwenGenerationSession、GenerationEpochLineage 和 DecodeStateIdentity 记录模型、前缀、position、epoch 及角色状态之间的关系。[D6]

22.2 流与终态

生成事件经 Core 的调用事件流传输；stable text decoding 需要处理 token/字节边界、特殊 token、最终 flush 和不能稳定输出的前缀。EOS、stop sequence、max tokens、失败、取消和 deadline 是不同结束原因。最终结果的归属不能由任意中间角色声明。

22.3 KV 复用

KV 不是普通不可变模型 artifact。可复用状态必须匹配模型内容与语义、adapter、runner、split、tokenizer、精确 prefix/长度、position、层范围、precision/layout、runtime ABI、Requester/安全域及有效期。过期、驱逐、重启或 pin 丢失不能继续作为命中证据。Provider session 提供 host/GPU pause、prefetch、pin、release 等显式状态操作。[D5, D6]

22.4 跨轮对话

ConversationStateBinding 和 NativeConversationCoordinator 用于绑定会话/轮次及状态来源，Python app journal 也保存请求和恢复记录。延续不是重放上一轮的未经验证对象；旧 attempt、晚到结果和已取消状态必须经过相应门禁。

验收边界 当前已有 tokenizer、epoch 和 conversation 组件；Spec182 的 sampling parity、stable epoch emission、journal/continuation 与完整原生 stream 接线仍保留未完成验收项。不把某些部件测试通过写成完整生成路径已迁移。

22.5 一次生成与下一轮对话

实际入口 runNativeEpochCoordinator 逐 role/epoch 调用 runner.run，模型状态机执行 Selecting→Prefilling→Decoding→Draining→Completed。attempt 是替换尝试，streamEpoch 是流实例，context epoch 是对话轮次，三者不按每个 token 同步递增。AC-18 给出方法状态表、两 token 示例、KV pin/迁移/释放以及 beginTurn→prepareCheckpoint→commitTurn 的完整阅读主线。

23 DI: 部署、扩展与管理接口

服务注册、制品准备、执行与管理动作各有生命周期

应用/部署层 app_sdk.application、deployment、controller 和 runtime_journal 管理定义、修订、请求引用、状态与恢复；显式部署可用于预热或兼容 workflow。

Provider 管理 APPProvider/ProviderAdminPort 提供 stage、activate、drain、delete、进度/检查点/输出报告；管理凭据与普通推理请求权限不是同一个授权入口。

原生 host NativeInferenceProvider 通过作用域注册句柄提供服务；共享 ExecutionLease 状态与 target 路由绑定物理槽位，关闭/代次阻断晚到回调。

扩展 SDK registry、loader、bounded executor 和 observer 注册策略/适配器；能力包含分割、放置、调度、准入、缓存、恢复、部署和执行调优等。

23.1 可选后端与工具

维护目录中有 ONNX、Qwen、YOLO adapter, Python 侧还保留 llama 等适配器与兼容入口；模型导出、图检查、GUI、ops 和 experiment tooling 不属于通用 Core。一个 API 导出不表示对应可选库、模型文件或 runtime 当前已安装。[D1, D8]

23.2 持久记录

部署修订、请求/attempt、计划摘要、Provider boot、grant、执行 lease 和结果引用需要一起定位，不能只靠一个 deployment 名称恢复。恢复时应按 owner 的契约重新验证仍有效的运行资源。

23.3 当前迁移状态

原生 Provider host/注册/共享 lease 已有进度记录；Python facade 兼容迁移、maintained caller 迁移及完整请求链仍未全部关闭。文档将 Python 控制路径和 C++ 组件同时列出，避免把 Spec182 的未来单一实现方向覆盖当前仍实际存在的 owner。[D7]

23.4 管理动作与请求动作的区别

stage 准备某部署修订，activate 使其可供请求，drain 限制新增工作，delete 处理该部署资源；这些管理接口由 APPProvider/ProviderAdminPort 拥有，不能用 NativeInferenceHandle.cancel 代替。恢复定位需要部署修订、request/attempt、plan、Provider boot 和有效 lease/grant，而非只有可读部署名称。新增 adapter/strategy 必须被 registry/配置实际选择，步骤见 AC-23。

24 UAV：服务容器、接口与操作界面

地面站与无人机都是可同时发起和提供服务的应用进程

UavDroneApp 组合飞控适配、视频、遥测、任务与内嵌 Repo；UavGroundStationApp 组合命令客户端、地图/任务界面、流订阅、记录播放和分析服务。主体代码拆在 Drone/GroundStationServiceContainer 与 Window 的 include 文件中；共享类型和命名位于 shared/。[U1, U2]

服务组	当前名称/后缀及作用
飞行命令	/UAV/MAVLink/Execute；受保护的 MAVLink 执行。
任务与状态	/UAV/Mission/Assign、/UAV/Telemetry/GetStatus；任务分配与遥测。
摄像头	/UAV/Camera/GetFrame、Provider 专属 /UAV/Camera/Video；取帧与开始/停止流。
录像与目录	Provider 专属 /UAV/Camera/Recording/Manifest、/UAV/Camera/Repo/Catalog。
参数与诊断	Provider 专属 Parameters、ParameterEdit、AnalyzeSnapshot 与 Preflight/Checklist。
识别与协作	/UAV/GS/ObjectDetection、/UAV/Incident/Analyze。
操作权管理	GS 的 OperatorAuthority/Lease、Revocation、Audit 服务。

名字可由 UavRuntimeConfig 配置；以上默认值不能代替部署配置和签名身份检查。

24.1 界面与状态模型

GUI 使用 typed TelemetryState、ReadinessState、FlightCommandState、MissionState、MissionProgressState、VideoState、AnalyzeSnapshot 和 dashboard 数据，区分无人机可达性、飞控 readiness、摄像头可用性、任务占用和视频会话。参数浏览/修改、preflight、目录查询和诊断由服务调用支撑，不直接绕过 Core 访问远端设备。[U2, U3]

应用边界 Core 传递 opaque MAVLink/应用 payload；飞行语义、地图与 GUI、媒体选择和检测模型均由 UAV 应用拥有。

24.2 界面如何使用状态

例如命令按钮关联一次 invocation 的 requestId，飞控 readiness 来源于遥测，视频窗口关联一个当前 stream session；三者分别更新。关闭视频窗口不应被解释为 mission 已取消，取得服务响应也不证明飞控 COMMAND_ACK 已到达。新增 UI 控件应订阅对应 typed state 并管理 Qt 回调生命周期，不直接重写设备状态。

25 UAV：飞控适配与操作权限

网络授权、操作权和飞行安全属于三个责任域，实际门禁位置须分别核对

25.1 飞控数据路径

GS 构造 MAVLink 命令，经 NDNSF 普通认证/token bootstrap 后对目标无人机使用 Targeted。Drone 的后端负责向 mock、UDP 或 serial 飞控路径传递字节并读取 ACK/遥测；mavlink-router 是部署连接方式之一，不是新的 Core 调用协议。[U2]

FlightControllerBackend 定义 sendMavlink、遥测、参数和 mission waypoint 接口。UDP backend 包含连接、帧解析、heartbeat、命令 ACK 和 mission upload 交互，并支持串口配置分支。GPS 信息来自飞控 MAVLink telemetry，而不是把任意本机 GPS 设备直接当作 NDNSF 身份可信来源。

25.2 三层权限与门禁

1. Core 验证 User/Provider 权限、签名、token、ControllerVersion 与请求绑定。
2. UAV 的 OperatorAuthorityLease 关联操作员、目标、范围和有效时间；issuer 提供申请、撤回、审计、持久化和续期逻辑。
3. GroundStation 的 flight/mission safety gate 使用新鲜遥测、readiness、armed/landed 等应用状态判断动作是否允许；lease 不能替代飞控本身的保护。

操作权 issuer 不等于 Core ServiceController；其 lease 修订也不是 Controller epoch。UI 阻止命令、网络拒绝授权和飞控返回 command ACK 必须分别显示。[U2, U3] 当前 Drone Execute handler 解码 command/mavlink_hex 后直接调用 backend，其函数体没有操作 lease 或 readiness 的独立重验；不能把 GS 侧门禁当作 Drone 侧同样强制执行的证据。AC-19 列出实际字段与 ACK 行为，TG-05 规划后端边界的明确校验。

25.3 现有支持与限制

mock/SITL、UDP、serial、遥测显示与参数操作为可运行入口；真实飞控、fail-safe、长时间断连恢复和户外设备条件仍需对应运行验证。代码具备硬件接口不表示本轮进行了飞行验证，也不等同于成熟飞控地面站的认证。

25.4 沿失败位置排查

命令尚未获得 Core 权限时停在服务入口；操作 lease 过期属于 UAV 操作权；设备链路断开或飞控拒绝属于 backend/飞控反馈。三类失败不可都显示为 timeout。AC-19 分开说明网络 Response、发送字节与 COMMAND_ACK，同时明确当前 Fields 命令表未形成覆盖全部命令的静态 schema；类型化补齐属于 TG-05。

26 UAV：巡逻任务、补偿与事件协作

长生命周期 mission 包含多个有限的 NDNSF invocation

26.1 巡逻分配

地面站将 waypoint 集合确定性分区/聚类, 形成任务 plan 与多个 part; 通过共享 /UAV/Mission/Assign 服务发现空闲 Drone。忙碌 Drone 可以发负 ACK 并报告 mission slot 占用; 应用根据候选选择, 而不是用 payload 里的 Provider 名称绕过协商。[U2, U4]

26.2 缺失部分的补偿

patrol task 关联 taskId、attemptId、partId、assignedProvider、deadline、状态和 response digest。某 part 超时或没有有效结果时, 下一次调用只补偿缺失部分; 每次仍有新的 Core requestId/token。任务业务“完成”需要所有必要 part 的有效结果, 而不是单次 invocation 有一个 Response。

26.3 任务与 job 状态

UavMissionSession 维护 planned、starting、active、degraded、compensating、recovering 到完成/取消/失败的任务状态, 并限制历史大小。每个 incident/job 另有 ACK 收集、ACK 关闭、计划提交、selected、evidence-ready、executing、reporting 和 terminal 状态。[U3, U4]

26.4 证据与终端归属

证据使用生产者精确命名 Data, 绑定 mission/incident、digest、时间与身份。终端报告必须匹配当前 attempt、request、planDigest 和选中的 terminal owner; 同一 incident 已接收的终态摘要不会被另一份晚到报告替换。恢复过程需要确认车辆与 stream 绑定, 不能只载入一个旧 ledger 就继续发飞行命令。[U4]

与 Core 的分工 Core 负责每次调用的认证与超时; UAV 负责 part 是否完成、哪些工作需要补偿、跨调用的任务身份以及任务层最终完成。

26.5 两个分片的任务例子

P0 已验证完成, P1 进入 missing 时, 仅对 P1 开启补偿 attempt; P0 的结果继续保留。新 invocation 获得新 token/assignment, 旧 P1 的迟到终端报告仍须匹配当前 attempt/plan/owner 才可接纳。恢复时先 reconcile 实际车辆与流, 再允许新的业务动作。AC-20 解释 bool/reason、局部状态与真实飞行停止的差异。

27 UAV：视频、遥测流与录像

小型控制请求建立会话，高频媒体通过命名 Data 传输

27.1 采集与呈现

VideoPublisher 使用 GStreamer 或兼容管线,支持摄像头/文件/测试来源。UavVideoPipeline 的 frame 结构携带 sessionEpoch、sourceFrameId、captureOriginNs、codec PTS、codecConfigEpoch、keyFrame 与编码字节；有界最新帧队列控制积压。readiness 报告真实来源及失败原因，fallback sample 不能当作真实摄像头采集证据。[U2, U5]

27.2 网络流与自适应

Provider 专属控制服务决定 start/stop；GS 订阅当前 live session，按 cursor/名称与元数据预测、取回和重排 Data。旧会话迟到包不进入当前视频。Core 提供通用窗口、名称映射、恢复与背压；UAV 根据帧类别、码率、画面时效和播放队列做媒体策略，并记录采集、到达、解码与呈现的不同时间点。

27.3 保密与留存

视频会话拥有受保护的 descriptor、密钥/nonce 与 Data 绑定。原始已签名 packet 可以进入 Repo；留存清单记录确切 packet、缺口和会话身份，不通过重新签名抹去原发布者。开始直播与是否开启 recording/retention 是不同控制条件；录像完成后通过 manifest/catalog 发现，按名字获取、验证、解密并播放。[U3, U5]

27.4 数据产品边界

当前具体 Repo-backed 产品主要是相机录像/packet 留存。Mission image、telemetry log、检测事件或报告虽可使用同样的 DataProductReference 模型，不能仅因类型存在就称所有产品均有完整采集和发布链。

时序边界 网络收到、解码完成与实际显示不是同一事件；低延迟或稳定性结论需要真实采集/呈现记录，不能用一个管线启动成功代替。

27.5 丢帧与录像完整性

最新帧队列中有 10、11、12 时 popLatest 只交付 12，其余计入 dropped。push 返回 true 也可能因为丢弃旧帧才腾出空间；超大单帧直接返回 false。这个显示策略不能复制到完整 packet 录像，后者需 manifest 和 gap 记录。AC-21 同时给出遥测单位、admit 多字段结果及 capture→传输→解码→显示/录像分工。

28 UAV：目标检测与多视角识别

有限识别 job 与长期任务分离，验证输入与结果的实际 owner

28.1 单视角辅助检测

Drone 可调用 GS 的 ObjectDetection 服务，请求只携带 frame/drone/目标类别等紧凑元数据，由 GS 对其已有的解码画面执行检测。YOLO worker 属于 UAV 应用侧计算工具；此调用不能自动证明使用了完整 NDNSF-DI 分布式规划路径。[U2]

28.2 多视角任务

MultiViewRecognitionJob 持有不可变精确视图引用、模型 profile、目标、mission/job/attempt 和时间边界。UavDetectorProvider 验证获取的 Data，并通过 IMultiViewRecognitionAlgorithm 接口执行。算法输出融合结果和 Provider-owned 标注引用；成功、视图不足、验证/关联失败、模型不支持、计算失败、发布失败、超时与取消分别编码。[U6]

28.3 真实模型与功能替身

C++ 有 DeterministicMultiViewAlgorithm 用于显式功能契约检查。真实 MVCNN-family ONNX 工具路径在 tools/mvcnn_onnx_worker.py：检查注册模型摘要、视图摘要和输入形状，显式要求 CPUExecutionProvider；images 与 viewMask 输入在图内联合处理，输出 logits 和 pooledFeatures。常规多视角 job 要求不同 UAV 的足够视图；单视角配对 baseline 不能冒充多视角融合。[U6]

28.4 不能扩大解释的结果

C++ 算法注入接口存在不证明真实 ONNX worker 已在每个默认 GUI/Provider 路径中接入；工具 fixture 的功能结果也不证明真实飞行准确率。跨进程获取、候选竞争、单一终端 owner、结果/标注发布和失败终态需在对应网络夹具中分别验证。

28.5 两视角输入到终端接受

两架 Drone 提供同一事件窗口的精确视图引用，job 绑定这些输入与模型 profile。executeMultiViewJob 的算法输出随后仍要经过 coordinator 的 job/attempt/terminalOwner 检查；算法成功不等于业务结果已接受。默认确定性算法、真实 ONNX worker 和单帧 YOLO 是独立入口。扩展点和例子见 AC-22。

29 跨模块协作的完整路径

用同一套 Core 机制连接领域任务与命名数据

29.1 UAV 录像路径

GS 经 Core 认证调用 Drone 的视频控制；Drone 发布受保护媒体 Data，可选内嵌 Repo 保存原始 packet 与留存 manifest。GS 查询录像清单，按精确引用取回并验证/解密后播放。UAV 定义媒体与保留语义；Repo 管字节/目录；Core 管调用、身份与传输。[U2, U5, R1]

29.2 DI 模型意图路径

应用提交模型和输入；Python 当前编排通过 Core 收集认证 ACK，adapter/placement 决定图有效的角色分配；可信物化后经 Repo 发布制品，再提交 Core Selection。Provider 原生运行器获取依赖、执行并发布输出，terminal role 产生被调用者接收的结果。原生 requester 仍未接通同等全链路。[D2, D5, D7]

29.3 UAV 分布式分析路径

任务管理器记录 incident 和精确 evidence；通过 Core 协作收集分析 Provider，固定角色/terminal owner，Provider 拉取并验证证据后返回报告及标注引用。若某服务选择以 DI 实现计算，应通过明确的 DI 应用接口接入；不能因同仓库有 DI 就把全部 UAV 检测自动画成 DI 调用。[U4, U6]

共同约束	具体要求
控制/数据分离	服务请求传小型描述和引用；完整对象/媒体沿合适的数据面传输。
身份与摘要	调用者、发布者、Provider、对象、request、attempt、plan 等按各层契约绑定。
状态不混淆	存储已提交、模型已准备、资源已预留、飞行已执行、画面已显示是不同事实。
恢复有 owner	Core 清理调用；Repo 恢复已验证字节；DI 管执行/会话；UAV 管任务补偿。

29.4 贯穿四模块的例子与实际接线边界

一次 UAV 事件可以组织为：相机经 Core 命名流发布画面；录像路径将原 packet 与 manifest 存入 Repo；地面站以精确画面引用构造分析 job；分析服务验证输入并返回绑定 job/attempt 的结果。若选择接入 DI，DI 还要完成模型准备、ACK 规划、角色执行与终端验证。前半段现有 UAV/Repo 入口与可选 DI 集成必须分开核对，当前原生 requester 的 NOT_READY 意味着不能把这条组合示例写成已经通过纯 C++ 端到端验收的调用链。

30 构建、配置与部署边界

源码、宿主构建、容器候选和运行证据分别建立身份

部署形态	组件与前提
本地 Core 应用	C++ runtime、ndn-cxx、NFD、ndn-svs、NAC-ABE、信任模式与私有身份环境。
Repo-node	Core 服务、SQLite/存储目录、热点缓存预算；大制品 backend 按所选物化方式配置。
UAV 应用	Drone/GS 容器、GUI/媒体依赖、mock 或实际 MAVLink 连接、摄像头、可选 Repo。
DI Provider	Core、原生 DI 库、匹配模型 runner、ONNX Runtime/可选 CUDA、精确模型与制品引用。
MiniNDN	多进程 NFD/路由/身份隔离、拓扑及实验脚本；不能用单进程调用替代网络证据。
Tiger/SIF	封存源码、容器内原生构建、候选身份、Apptainer/Slurm、实际节点设备和执行记录。

30.1 工具链一致性

宿主原生构建按仓库规则使用 system compiler/binutils 和匹配的 Boost 头/库，先完成 NAC-ABE 依赖闭包，再构建 Core 与 Python 扩展。改变 C++ ABI 后，需要重新构建依赖对象并核对实际加载的库；导入成功和 SONAME 不变都不能独立证明 ABI 一致。[O1]

30.2 容器与集群

Tiger 的唯一维护入口为 Experiments/TigerCluster/。适配器、job/profile、images 和 results 按该目录契约组织；共享组件仍由 packaging 维护。镜像原生组件须在容器 builder 内编译，不使用宿主 Python.h、venv 或宿主.so 替代镜像依赖。开发机负责代码/静态检查/测试，实验机负责 SIF 构建和 Tiger 实验。[O2]

30.3 配置、身份和数据

私钥、模型、数据库、缓存、运行日志与镜像不因文档生成进入版本库。模型名、设备参数和服务路由需与实际被加载/注册的内容核对；本轮没有启动 NFD/MiniNDN、构建原生依赖或向集群提交任务。

30.4 可复现入口与本轮命令

本轮文档命令从仓库根执行：python3 Design/build-api-reference.py；python3 Design/build-behavior-coverage.py；python3 Design/refresh-snapshot.py；python3 Design/render-api-contracts.py -target；python3 Design/build.py；python3 Design/verify.py 后接本次输出目录。原生编译参数按

docs/native-build-parallelism.md 与当前 Spec 的 quickstart/验证脚本，镜像按 Experiments/TigerCluster/docs/sif-build.md。文档更新不自行启动原生构建、SIF 或集群作业。

31 实现状态与验证口径

把已有代码、接线状态和运行资格分开报告

范围	本版可支持的结论
Core	已有通用调用、Targeted、协作、流、安全及在线授权组件；本文核对主要生产路径与状态分支。
Repo	已有 C++ 对象存储/服务、Python 编排和制品 API；具体后端、目录操作及控制模式存在差异。
UAV	已有飞控/GUI/任务/视频/Repo/识别组件和研究夹具；真实硬件和模型接线按入口分别确认。
DI	C++ PreparedModel/NativeInferenceClient 与 Provider 执行链已接通并由原生过程矩阵验证；旧 Python 编排、subinterpreter/wheel 与外部资格仍分别保留边界。
本轮文档	源码核对、PDF 构建与文本/版面检查；没有新协议测试或性能测量。

31.1 测试入口

Core 检查位于 tests/unit-tests/ 与 tests/integration-tests/，包含 controller-revocation、controller-version-refresh、invocation-stream、stream-facade、opaque-selection 等主题。Repo 有存储后端/制品传输 C++ 检查与 test_spec164_* Python 检查；UAV 有 mission/collaboration/multiview 检查和 tools 运行夹具；DI 有 di-native-* 及 Python/experiment 检查。[V1]

这些名字是继续验证的入口，不是本轮通过清单。测试文件存在、一个 fixture 运行成功、完整网络结果和指定候选的硬件资格验收是不同证据等级。

31.2 活跃实现

当前 active Spec 为 185-prepared-model-runtime；其 C++ qualification、Python binding 和 Design handoff 证据控制本轮状态。Spec184 的 Qwen3.6-27B、继承 negative/retirement、I05、Python retirement 与 SIF/Tiger 资格仍由 specs/184-native-di-closure/ 独立控制，本文不改写这些任务的验收状态。

失效快照的处理 代码变化后，source-baseline 摘要可能与当前工作树不再一致。这表示设计快照需要重新核对，而不是证明新代码有缺陷；目标文档也不能凭自动复制变成实现事实。

31.3 能力、入口与证据的对应

Core 在线权限以 ServiceController::grant/revoke 和第 9–13 章分支为依据；Repo 精确目录以 RepoCore::catalogLookup 为依据；UAV 最新帧以 BoundedLatestFrameQueue::push/popLatest 为依据；DI 请求句柄以 NativeInferenceClient/PreparedModel 为依据。这些行为核对分别进入 AC-08、

AC-13、AC-21、AC-14。生成/KV、原生过程、会话恢复和 Python facade 的当前证据分别链接 Spec185 B7/B8；Spec184 外部模型、retirement 与 SIF/Tiger 资格仍保持独立。Design/reviews 的修订记录只证明文档工作，不替代这些功能门。

32 源码索引：Core 与授权

按入口定位，再用快照摘要确认正在阅读的版本

下列路径以仓库根为起点；正文中的引用 ID 指向这里和下一节。符号用于定位职责；详细文件清单与 SHA-256 见 source-baseline.json。

编号	规范文件与关注点
C1	docs/architecture.md; docs/ndnsf-core-app-boundary.md, 总体边界；以源码修正旧叙述。
C2	ndn-service-framework/ServiceContainer.hpp、LocalServiceRegistry.hpp；同目录 ServiceController/ServiceProvider。
C3	Core 的 ServiceUser.hpp、ServiceUser.cpp、ServiceProvider.cpp、NDNSFMessages.hpp；调用与协作。
C4	Core 的 GenericSelectionTxnStore.hpp、GenericSelectionTxnStore.cpp；不透明选择事务。
C5	Core 的 ServiceUser/ServiceProvider、RequestConfidentiality.cpp、HybridMessageCrypto.cpp；大数据和加密。
C6	Core 的 Stream.hpp、Stream.cpp、StreamFacade.hpp、StreamFacade.cpp；连续流、预测、FEC。
C7	Core 的 InvocationStream.hpp、InvocationStream.cpp；调用事件流与终态。
C8	Core 的 ExecutionLease.hpp、NetworkTelemetry.hpp、TimelineTrace.hpp 及 Provider；资源和观测。

32.1 授权引用

S1: ControllerVersion.cpp 的 isValid/compare。S2: ServiceController.cpp 的 getPolicyStatus。S3: 同文件的 reissueAbePolicies、rotateAbeGenerationAndReissuePolicies、reconcilePendingAbeRotation、advanceAuthorizationEpoch、revoke。S4: 同文件的 grant。S5: User/Provider 的 applyPermissionResponse。S6: installControllerStatus。S7: invalidateControllerScopedCaches 与 Provider 的 refreshProviderPermissionsAfterAdvance。S8: Controller 的 onPolicyStatusInterest、User/Provider 的 fetchPolicyStatusFromController/onPolicyStatusData。S9: PolicyRefreshCoordinator.cpp。S10: refreshNacDkeyForControllerStatus。S11: scheduleControllerStatusRefresh 与周期配置。S12: restorePersistedRuntimeStatuses。[均位于 Core 目录]

33 源码索引：Repo 与 DI

文件清单包含规范入口；不以临时 `compare/build` 副本替代当前源文件

编号	路径（每组根目录见下文）及关注点
R1	Repo README_ch.md、include/ndnsf-distributed-repo/RepoCore.hpp；对象与边界。
R2	Repo src/RepoNode.cpp、src/RepoClient.cpp；服务与控制入口。
R3	Repo src/RepoTypes.cpp；普通 SQLite/热点缓存后端。
R4	Repo include 中 RepoStoreBackend.hpp、FilesystemArtifactStore.hpp、ArtifactTransfer.hpp、ArtifactManifest.hpp。
R5	Repo pythonWrapper/py_repoclient/artifact_api.py；公开会话 API。
R6	同 Python 包 network_artifact_backend.py；真实网络发布、协作和 receipt 绑定。
R7	同 Python 包 orchestration.py、persistence.py 与 Repo src/RepoCore.cpp；目录、副本与后端。
D1	DI Python api/__init__.py、app_sdk/client.py、application.py、provider.py。
D2	DI Python app_sdk/placement.py、sdk/placement.py；ACK 驱动规划与 V3。
D3	DI C++ NativeRequestPreparation.cpp、NativeOfferAdmission.cpp、NativeV3Placement.cpp。
D4	DI C++ NativePlanSealer、NativeGrantClient/Verifier、ProtectedRuntime、NativeProtectedArtifactStore；封装和保护。
D5	DI C++ NativeProviderSession/Runtime/Handler、ProviderRoleWorker、TensorBundleCodec、DependencyWaitScheduler、CollectiveRuntime。
D6	DI C++ NativeEpochCoordinator、NativeConversationCoordinator、NativeStandaloneTokenizer；生成与会话。
D7	DI C++ PreparedModel.cpp、NativeInferenceClient.cpp: prepared request/dispatchOperation；Spec185 tasks 与 evidence。
D8	DI C++ NativeInferenceProvider；Python app_sdk 与 sdk 的部署/扩展接口。
Repo 根为 NDNSF-DistributedRepo/；DI Python 根为 NDNSF-DistributedInference/ndnsf_distributed_inference/，DI C++ 根为 NDNSF-DistributedInference/cpp/ndnsf-di/。	

33.1 生成与会话的准确文件入口

D6 的生成自由函数位于 NDNSF-DistributedInference/cpp/ndnsf-di/NativeEpochCoordinator.cpp；QwenGenerationSessionStateMachine 的实现位于 cpp/adapters/qwen/QwenGenerationSession.cpp；会话日志位于 cpp/ndnsf-di/NativeConversationCoordinator.cpp；缓存状态机在 NativeProviderRuntime.cpp 的 ConversationStateStore。由 AC-18 的所属符号和签名定位，不把同名转发头当完整实现。

34 源码索引：UAV 与交付

应用服务、硬件/模型工具与运行入口分别定位

编号	规范路径与关注点
U1	UAV README_ch.md; shared/UavNames.hpp。
U2	UAV drone/DroneServiceContainer.inc.hpp、ground-station/GroundStationServiceContainer.inc.hpp 及各 App/Window。
U3	UAV shared/UavProtocol.hpp; 状态与线协议类型。
U4	UAV shared/UavMissionSession.cpp、ground-station/UavIncidentCoordinator.cpp、drone/UavCollaborationParticipant.cpp。
U5	UAV shared/UavVideoPipeline.hpp/.cpp 及 Drone/GS 容器; 媒体与留存。
U6	UAV shared/UavMultiViewRecognition.cpp、UavDetectorProvider.cpp、tools/mvcnn_onnx_worker.py。
O1/O2	AGENTS.md、docs/native-build-parallelism.md、Experiments/TigerCluster/README.md 与 docs/sif-build.md。
V1	根目录 tests/、Experiments/、各模块 examples/tools 及 active Spec evidence。

UAV 根为 NDNSF-UAV-APP/。O2 中的 docs/sif-build.md 相对 Experiments/TigerCluster/; 其他交付路径相对仓库根。

34.1 模块维护文件清单

module-inventory.json 按 Core、UAV、DI、Repo 以及公共绑定/支持目录记录受版本库维护的文件路径、类型、大小及源码摘要; 模型、原始日志和构建目录不因文档任务被复制进 Design。重点核对的规范文件另列于 source-baseline.json。

34.2 源码检索口径

CodeGraph 用于定位规范符号和调用者; 出现 compare-old/compare-new、staging 或 build 副本时, 以仓库维护的原始路径为准。章节中的运行限制来自具体实现分支, 不是根据类名或 README 的目标描述推测。

34.3 索引的使用范围

UAV 高频回调和硬件行为常在.inc.hpp 与.cpp 中, 公开头文件清单不能代替这些实现。video queue 查 shared/UavVideoPipeline.cpp, mission 查 shared/UavMissionSession.cpp, 实际命令解析查 drone/DroneServiceContainer.inc.hpp。module-inventory.json 是 R0 历史盘点, 当前新增/删除检测以 design_state.source_files 和 source-baseline 为准。

35 文档维护与后续目标变更

当前设计随已核对代码更新；目标设计随用户明确决定更新

1. 当前正文保存在 `current-content.tex`，目标正文保存在 `target-content.tex`。首版完全相同；以后两份独立维护，不用共享正文把目标修改自动传播到当前设计。
2. 修改目标时，记录变更动机、涉及模块、与当前行为的差异、依赖和验证条件。未决定的方案保持待讨论，不把想法当作已接受契约。
3. 实现落地后，重新核对生产路径、状态/安全分支和实际证据，更新当前正文与源码快照；不能仅因为目标 PDF 已更新就宣称功能完成。
4. 运行 `python3Design/build.py` 重建两份 PDF，检查文本、分页、字体、表格和图。当前与目标按各自源码身份检查；TG-01 至 TG-05 的有意差异必须保留。
5. 更新 `CHANGELOG.md`、`validation.md` 和相关 Spec 进度。本目录完整纳入 Git；每个 Spec 的设计影响记入 `spec-design-changes.md`，与正文和 PDF 同步提交。

35.1 覆盖清单与源码快照

`coverage-matrix.md` 记录四模块各功能域对应章节与主要入口；`module-inventory.json` 保留 R0 历史盘点；新增/删除由当前源码范围检查发现。文件被列入清单不表示每一行都完成审计；正文区分主要流程源码核对与配置/工具入口登记。

`source-baseline.json` 记录重点核对文件的摘要、采样时提交和工作树差异。本地源码快照与 PDF 构建日志位于唯一的 `.codex-tmp/` 目录，供核对文档所描述的时点；它们不授予任何运行资格。

35.2 本文未引入的行为变化

本版不新增 SVS epoch 公告、不改变 grant 的本地失效范围、不补接原生 DI 请求链，也不扩大 Repo/UAV 现有接口能力。目标 TG-01 至 TG-05 已登记改进方向；下一步沿 Spec182 推进 TG-02，并逐项定稿其余目标的接口和验收条件。

交付标准 两份完整中文 PDF、可编辑源文件、覆盖/源码记录和验证记录一起维护。文档完成与代码实现完成是两个独立状态。

35.3 章节的人工接受条件

读者应能从每个关键入口说明参数从哪里来、下一步调用什么、怎样判定完成以及失败保留在哪一层。签名生成和 PDF 校验无法自动证明这些问题已回答。修改需连同 API、正文、目标差异、Spec 记录、PDF 和验证证据形成一个本地提交；未定义的字段/线程保证明确登记，禁止用增加页数替代行为契约。

36 API 阅读方法与覆盖边界

AC-01 · Core 接口契约

36.1 阅读路径与例子的前提

先按前 35 章找到组件，再按 AC 编号查本节契约。每个签名上方给出所属符号，避免把两个类同名的 cancel 当成同一操作。下方的参数、状态和例子解释如何使用。例子省略证书、Face、配置和业务序列化的建立，不是可直接启动的完整程序；具体类型和重载用 API ID 到 api/ 声明参考查找。

36.2 常用术语

项目	行为与调用含义
owner	负责创建、修改和销毁某项状态的组件；不是仅持有一个指针的调用方。
ACK_CLOSED	ACK 收集结束后冻结的候选快照。ackClosedDigest 将规划输入绑定到这一次观察；不能换成模型摘要。
proposal / plan core	proposal 是待验证的放置建议；plan core 是封存的执行内容。产生建议不等于获准执行。
projection / grant view	projection 是给特定参与者的选择信息；grant view 是授权计算需要的计划视图；两者不是任意可互换 JSON。
recipe / exact reuse	recipe 描述怎样装配实际制品。exact reuse 要求身份、角色、设备和有效期均匹配，不能只看 has_model。
epoch / lineage	epoch 必须带所属域理解；lineage 关联生成前缀与执行轮次。Controller、Repo、attempt、stream、conversation 的计数不能直接比较。
观察状态	SIGNATURE_ONLY 仅有声明；CONTRACT_REFERENCED 有人工说明引用，仍不等于全部运行行为已验收。

36.3 一次查阅示例

需要停止生成时，先查 AC-14 的请求句柄 cancel，再查 AC-18 的模型状态机和 Provider KV 操作。句柄终态取消可无操作，而 Qwen 状态机终态再次 cancel 会抛异常；KV release 是另一项显式资源动作。这样的差异必须由所属符号和状态契约判断，不能由动词猜测。

完整声明查询：api/core-reference.md。

37 Core API: 容器与角色生命周期

AC-02 · Core 接口契约

37.1 创建与所有权

ServiceContainer 组合进程中的角色。RuntimeConfig 的 containerIdentity 是容器身份, groupPrefix 是组命名范围, controllerPrefix 指向权限控制器, trustSchemaPath 指定信任规则。addController/addUser/addProvider 登记角色; use 系列使用外部已有对象, 外部对象必须比容器的相关使用期更长。本地 addLocalService 的 RequestT/ResponseT 是 C++ 类型约定, 不产生网络服务权限。

37.2 调用次序

```
// 示意: roles/config/handlers 已由应用建立
// 1. 建立身份、Face 与配置
// 2. 登记 Controller/User/Provider 与本地服务
// 3. 登记启动/停止 hook
container.start();
// 4. 应用运行 Face 事件循环, 处理服务回调
container.stop();
// 5. 排空应用 worker 后释放借用的角色、Face 和回调资源
```

37.3 前置条件与失败

受保护的注册方法调用 ensureNotStarted, 启动后修改会被拒绝; 空角色对象和缺失角色是本地配置错误, 不是网络超时。start/stop 的 hook 不能代替应用 worker 的排空。容器本地调用不经过网络权限握手, 只有已经可信的进程内调用方可使用。扩展应把模型加载、数据库连接和飞控 worker 放在应用 hook 中, Core 容器不解释这些领域状态。

37.4 完整签名与源码定位

API-ee68261266a4 public

所属符号: ndn_service_framework::ServiceContainer::addLocalService

```
template <typename RequestT, typename ResponseT, typename HandlerT>
void
addLocalService(const ndn::Name& serviceName, HandlerT&& handler)
```

源码: ndn-service-framework/ServiceContainer.hpp, 第 73 行。

API-04ad67003514 public

所属符号: `ndn_service_framework::ServiceContainer::start`

```
void
  start()
```

源码: `ndn-service-framework/ServiceContainer.hpp`, 第 331 行。

API-cd596b18ee08 public

所属符号: `ndn_service_framework::ServiceContainer::stop`

```
void
  stop()
```

源码: `ndn-service-framework/ServiceContainer.hpp`, 第 365 行。

完整声明查询: `api/core-reference.md`。

38 Core API: 请求发起与 Targeted 调用

AC-03 · Core 接口契约

38.1 请求标识与参数

普通 `RequestService` 的 `serviceName` 表示服务, `requestMessage` 是序列化请求, `timeoutMs` 是毫秒预算, `strategy` 默认 `FirstResponding`。另一组重载额外提供 ACK 窗口与候选回调。`RequestServiceTargeted` 额外指定 `provider`。返回 `ndn::Name` 是请求标识, 后续结果经 `ResponseHandler` 交付; `TimeoutHandler` 处理超时, 不是返回值的一部分。保持回调捕获资源有效, 并按 `User` 的 `Face/I/O` 执行上下文调用。

38.2 最小调用示意

```
// user、RequestMessage payload、onTimeout/onResponse 已建立
auto requestId = user.RequestService(
    ndn::Name("/example/echo"), payload, 3000, onTimeout, onResponse);
// 保存 requestId 用于观测; 此处尚未取得业务响应
// 已知 provider 的调用使用 RequestServiceTargeted(provider, ...)
```

38.3 事件与接受边界

项目	行为与调用含义
Request → ACK	Provider 验证发现请求并报价, ACK 说明候选愿意参与; 一般 ACK 不保证已独占预留资源。

项目	行为与调用含义
ACK → Selection	User 按候选策略选择，绑定请求与 token；未被选中的 Provider 不执行请求。
Selection → Response	Provider 验证选择后执行；User 验证响应后才交付。拿到 requestId 或收到 ACK 都不能记为完成。
Targeted	已有认证 token 关联时走短路径；token 不可用时遵循 bootstrap/失败分支，不跳过授权。
错误观察	无 ACK、权限/响应验证失败、业务错误与 deadline 是不同边界。getRequestStatus 是本地状态查询，不能替代远端实际执行证据。

38.4 回调类型

低层响应回调收到 ResponseMessage 的常量引用，超时回调收到请求 Name；要异步保存结果就复制所需数据，不捕获一个仅在回调期间有效的引用。类型化重载另行解码 ResponseT。

```
using ResponseHandler = std::function<void(const ResponseMessage&);>;
using TimeoutHandler = std::function<void(const ndn::Name&);>;
```

38.5 完整签名与源码定位

API-09fc656cb649 public

所属符号：ndn_service_framework::ServiceUser::RequestService

```
ndn::Name RequestService(const ndn::Name& serviceName,
                        ndn_service_framework::RequestMessage requestMessage,
                        int timeoutMs,
                        TimeoutHandler onTimeout,
                        ResponseHandler onResponseHandler,
                        size_t strategy = ndn_service_framework::tlv::FirstResponding);
```

源码：ndn-service-framework/ServiceUser.hpp，第 802 行。

API-1009e77682c2 public

所属符号：ndn_service_framework::ServiceUser::RequestServiceTargeted

```
ndn::Name RequestServiceTargeted(const ndn::Name& provider,
                                const ndn::Name& serviceName,
                                ndn_service_framework::RequestMessage requestMessage,
                                int timeoutMs,
                                TimeoutHandler onTimeout,
                                ResponseHandler onResponseHandler);
```

源码：ndn-service-framework/ServiceUser.hpp，第 795 行。

API-efa9e29aae17 public

所属符号: ndn_service_framework::ServiceUser::RequestServiceTargeted

```
template<typename RequestT, typename ResponseT>
ndn::Name RequestServiceTargeted(const ndn::Name& provider,
                                const ndn::Name& serviceName,
                                const RequestT& request,
                                std::function<void(const ResponseT&)> onResponse,
                                std::function<void()> onTimeout,
                                int timeoutMs)
```

源码: ndn-service-framework/ServiceUser.hpp, 第 1082 行。

完整声明查询: [api/core-reference.md](#)。

39 Core API: Provider 注册与作用域回收

AC-04 · Core 接口契约

39.1 注册者必须保存句柄

addScopedService 的 ackHandler 处理发现阶段资格, requestHandler 处理被选定后的请求; addScopedCollaborationHandler 的 handler 接收协作角色与 assignment。返回的 ServiceRegistration 是 move-only 句柄, 不能当临时返回值立即丢弃, 否则作用域回收会关闭这一代注册。模板服务输入/输出必须满足对应序列化契约。

39.2 调用与关闭示意

```
// service、ackHandler、requestHandler 和模式参数采用上方重载
auto registration = provider.addScopedService(
    service, ackHandler, requestHandler /* 其余参数见签名 */);
// 保存 registration, 运行事件循环
registration.close();
// Provider generation 阻止这一代新的 ACK/Selection/执行工作
```

39.3 时序与资源

在 Face 线程或事件循环开始前注册。Request 认证通过才触发 ACK, Selection 绑定通过才触发执行; 不要在 ACK 回调里提前执行有副作用的业务。close/析构先关闭 generation, 再投递清理; Provider 已销毁时旧句柄关闭为无害操作。正在运行的外部业务 worker 仍须自行停机/排空, 关闭服务不会删除网络缓存中的历史 Data。legacy 注册不能覆盖仍存活的 scoped owner。

39.4 完整签名与源码定位

API-c3a3fd9da1b0 public

所属符号: `ndn_service_framework::ServiceProvider::addScopedService`

```
ServiceRegistration addScopedService(const ndn::Name& serviceName,
                                     AckStrategyHandler ackHandler,
                                     RequestHandler requestHandler,
                                     ServiceInvocationMode invocationMode);
```

源码: `ndn-service-framework/ServiceProvider.hpp`, 第 860 行。

API-23e6ea05d4d6 public

所属符号: `ndn_service_framework::ServiceProvider::addScopedService`

```
ServiceRegistration addScopedService(const ndn::Name& serviceName,
                                     AckStrategyHandler ackHandler,
                                     RequestHandler requestHandler,
                                     ServiceMode mode);
```

源码: `ndn-service-framework/ServiceProvider.hpp`, 第 864 行。

API-33314e8f5bbb public

所属符号: `ndn_service_framework::ServiceProvider::addScopedCollaborationHandler`

```
ServiceRegistration addScopedCollaborationHandler(
    const ndn::Name& serviceName,
    std::vector<CollaborationRole> allowedRoles,
    AckStrategyHandler ackHandler,
    CollaborationHandler handler);
```

源码: `ndn-service-framework/ServiceProvider.hpp`, 第 873 行。

完整声明查询: [api/core-reference.md](#)。

40 Core API: 延迟规划与协作提交

AC-05 · Core 接口契约

40.1 两种时间预算

`ackCollectionTimeMs` 只控制报价收集窗口, `timeoutMs` 控制整个请求期限。例如总预算 5000 ms、ACK 窗口 1000 ms, 报价结束后准备与执行共用剩余预算, 不能再获得全新的 5000 ms。 `onAckClosed` 给出不可变候选闭包, 应用此时才决定本次角色和参与者。

40.2 两角色例子

设 role A 在 Provider P1 产生中间对象, role B 在 P2 消费该对象并返回终端结果。应用计划需表达两个角色的参与者、A→B 的依赖及其命名/密钥作用域、终端接收规则; Core 认证和传递这些内容, DI 决

定张量是否合法, UAV 决定任务分区是否合法。两个角色不是两个独立、互不绑定的 RequestService。

40.3 提交顺序与结果

```
// 回调顺序示意, 不展开应用计划编码
BeginCollaboration(..., onAckClosed, onFinalResponse, onTimeout);
// onAckClosed(snapshot):
//   plan = 按 snapshot 校验候选并构造应用计划
//   accepted = CommitCollaborationPlan(requestId, snapshotDigest, plan)
// accepted 只表示本地提交被接受; 等待最终结果或失败
```

40.4 不可越过的状态

错误 requestId、闭包摘要不匹配、请求过期或状态不允许提交时, 不得把失败提交继续显示为执行中。bool 不是各 Provider 的完成聚合值。GenericSelectionTxnStore 的 prepare/commit 保存 Provider 侧认证选择和 opaque blob, 与 User 的 CommitCollaborationPlan 是两个 owner; 恢复事务还需匹配 Provider boot epoch、token/lease 和期限, 不能用旧 WAL 绕过新进程验证。

40.5 CollaborationPlan 的具体字段

项目	行为与调用含义
窗口/期限	ackCollectionTimeMs 默认 200, timeoutMs 默认 5000, 单位毫秒; 与 BeginCollaboration 的请求预算一致管理。
roles	每项含 role/service/requiredArtifact、allowDynamicProvisioning=false、provisioningTimeoutMs=30000、appRequirement/assignmentPayload、minProviders/maxProviders 默认 1、terminalResponseOwner=false。流式协作指定唯一用户终端 owner。
keyScopes	每项含 name 与允许的 roles; 依赖引用该密钥作用域, 而不是自行传递一个未绑定的解密密钥。
dependencies	每项含 producers、consumers、keyScope、topicPrefix、required=true; A→B 的例子分别在 producers/consumers 填 A/B。
sharedAssignmentMetadata	选中参与者共享的不透明 metadata, 与各角色 assignmentPayload 分开传递。
participantSelector	shared_ptr<const ParticipantSelectionPolicy>; select(candidates, roles) 返回 SelectedParticipant, 含精确 Provider、角色、assignment 与实际 ACK 绑定。

40.6 完整签名与源码定位

API-c9045b5b03e6 public

所属符号: `ndn_service_framework::ServiceUser::BeginCollaboration`

```
ndn::Name BeginCollaboration(const ServiceName& service,
                             const RequestPayload& initialRequest,
                             int ackCollectionTimeMs,
                             int timeoutMs,
                             CollaborationAckClosedHandler onAckClosed,
                             ResponseHandler onFinalResponse,
                             TimeoutHandler onTimeout,
                             const RequestId& requestId = RequestId());
```

源码: `ndn-service-framework/ServiceUser.hpp`, 第 935 行。

API-3ab0d3d87607 public

所属符号: `ndn_service_framework::ServiceUser::CommitCollaborationPlan`

```
bool CommitCollaborationPlan(const RequestId& requestId,
                             const std::string& ackClosedDigest,
                             CollaborationPlan plan);
```

源码: `ndn-service-framework/ServiceUser.hpp`, 第 985 行。

完整声明查询: [api/core-reference.md](#)。

41 Core API: 调用流的事件和终态

AC-06 · Core 接口契约

41.1 如何取得 writer

User 通过 `RequestServiceStreaming` 建立单次调用并保存共享 handle; Provider 通过 `addStreamingHandler` 注册流处理器, 由该处理器获得 `StreamedResponseWriter`。不要自行构造脱离 Core 请求绑定的 writer core。onEvent 接收 `EventT`, onComplete 接收最终 `ResponseT`, onError 接收 `StreamedInvocationError`; 选定的模板类型需要可序列化。

41.2 返回值与异常

项目	行为与调用含义
<code>publish(event) → bool</code>	模板先序列化再调用 core; 任何异常被捕获并返回 false。false 也可能是 core 拒绝发布, 不能仅解释为队列已满, 更不能等同远端消费完成。
<code>finish(response, reason)</code>	返回 void; core 拒绝时抛 <code>logic_error</code> (stream finish rejected); 序列化异常也可传播。不能在 finish 后继续发布普通事件。

项目	行为与调用含义
fail(code, message)	错误终态已被占用时抛 logic_error；与 publish 的吞异常返回 false 行为不同。
取消/期限	isCancelled 与 remainingDeadline 用于工作边界检查, 仍需处理检查后发生取消的竞争；最终状态由 Core 仲裁。
有界流	options 管理期限、排队、保留与重试。事件留存可重传不等于业务执行可重复；应用不应通过再次执行生成来补一个遗失的网络事件。

41.3 两事件示意

```
// 在已认证的 Provider streaming handler 中
if (!writer.publish(event1)) { /* 停止产生新事件并检查关闭原因 */ }
else if (!writer.publish(event2)) { /* 同上 */ }
else { writer.finish(finalResponse, finishReason); }
// 必须处理 finish 的异常；不能假设 publish(false) 从不发生
```

41.4 完整签名与源码定位

API-9e9c476a768e public

所属符号: ndn_service_framework::ServiceUser::RequestServiceStreaming

```
template<typename RequestT, typename EventT, typename ResponseT>
std::shared_ptr<StreamedInvocationHandle<EventT, ResponseT>>
RequestServiceStreaming(
    const ndn::Name& serviceName,
    const RequestT& request,
    StreamedInvocationOptions options,
    std::function<void(const EventT&)> onEvent,
    std::function<void(const ResponseT&)> onComplete,
    std::function<void(const StreamedInvocationError&)> onError,
    size_t strategy = ndn_service_framework::tlv::FirstResponding)
```

源码: ndn-service-framework/ServiceUser.hpp, 第 856 行。

API-2eccc93af394 public

所属符号: ndn_service_framework::StreamedResponseWriter::publish

```
bool publish(const EventT& event)
```

源码: ndn-service-framework/InvocationStream.hpp, 第 348 行。

API-2bdd4eb83edb public

所属符号: `ndn_service_framework::StreamedResponseWriter::finish`

```
void finish(const ResponseT& response, StreamFinishReason reason)
```

源码: `ndn-service-framework/InvocationStream.hpp`, 第 359 行。

完整声明查询: `api/core-reference.md`。

42 Core API: 命名对象与连续流

AC-07 · Core 接口契约

42.1 选择哪一种 API

项目	行为与调用含义
签名对象	<code>publishSignedAppData</code> 发布 <code>dataName</code> 对应的字节, <code>freshness</code> 是毫秒; <code>fetchSignedAppData</code> 传精确名字、 <code>expectedSigner</code> 、 <code>timeoutMs</code> 和成功/失败回调。验证签名者后才能消费。
完整大对象	<code>publishEncryptedLargeData</code> / <code>fetchEncryptedLargeData</code> 或协作中的 <code>publishLargeNamed</code> / <code>fetchLargeExact</code> 处理分段与完整对象。 <code>manifest</code> 引用和实际对象摘要由应用解释。
连续流	<code>StreamPublisher::push</code> 接收 <code>shared_ptr<ndn::Data> signedData</code> , 调用前已构造并签名; 它不是接收未签名业务块并自动解释视频 <code>metadata</code> 的接口。
调用事件流	AC-06 的 <code>writer</code> 接收 <code>EventT</code> , 由 Core 绑定单次请求、序号和终态。不能把它当成无结束时间的摄像头会话。

42.2 连续流例子

```
// signedData 已按流的精确名称/会话规则构造并签名
publisher.push(signedData); // void, 不返回逐包送达确认
// subscriber 按会话与 cursor 获取并验证
// 应用决定过期视频帧可丢弃, 完整录像对象必须完整验证
```

42.3 资源与恢复

发布窗口、订阅缓存和恢复范围有界, FEC 修复的是不透明字节, 不能代替媒体解码或完整对象校验。发布、签名验证、重排、业务消费是不同阶段。调用者保存异步回调依赖的资源; 流 `stop` 后历史 `Data` 仍可能在缓存中, 应用必须用会话身份拒绝迟到旧帧。

42.4 完整签名与源码定位

API-160499aab055 public

所属符号: `ndn_service_framework::ServiceUser::publishSignedAppData`

```
ndn::Name publishSignedAppData(  
    const ndn::Name& dataName,  
    const ndn::Buffer& payload,  
    ndn::time::milliseconds freshness = ndn::DEFAULT_FRESHNESS_PERIOD);
```

源码: `ndn-service-framework/ServiceUser.hpp`, 第 714 行。

API-ef2281d2fe2d public

所属符号: `ndn_service_framework::ServiceUser::fetchSignedAppData`

```
void fetchSignedAppData(  
    const ndn::Name& dataName,  
    const ndn::Name& expectedSigner,  
    int timeoutMs,  
    SignedAppDataHandler onData,  
    SignedAppDataFailureHandler onFailure);
```

源码: `ndn-service-framework/ServiceUser.hpp`, 第 737 行。

API-68706878d1a8 public

所属符号: `ndn_service_framework::StreamPublisher::push`

```
void push(std::shared_ptr<ndn::Data> signedData);
```

源码: `ndn-service-framework/StreamFacade.hpp`, 第 46 行。

完整声明查询: [api/core-reference.md](#)。

43 Core API: 在线授权与版本状态

AC-08 · Core 接口契约

43.1 输入是单项授权

`grant(identity, serviceName, authorizationAttribute)` 接受一次身份/服务/属性操作。控制器把服务加入已有 `allowedServices`, 把属性加入集合, 再从有效属性重建完整 ABE 策略; 调用者不提交该身份的全量策略。`grant-only` 推进 `ControllerVersion`, 通常不旋转 ABE generation; 如已有 `pending rotation`, 先协调恢复, 因此不能保证每次 `grant` 调用都绝无全局轮换。

43.2 返回分支

项目	行为与调用含义
grant 返回 false	可能是非法 target、generation 未就绪、待轮换恢复失败、证书已撤销，或相同授权已经存在且无需重授的 no-op。bool 不能唯一编码失败原因。
grant 产生变更	更新目标身份的权限与属性，移除相应服务撤销/可恢复的身份撤销；证书撤销不能靠 grant 覆盖。
revoke(target)	RevocationTarget 显式区分身份、服务授权和证书。撤销事实/版本持久化后再做 ABE 轮换；后续失败不意味着撤销自动回滚。
getPolicyStatus(service)	得到服务相关撤销信息与全局版本；发布的外层 Data 要签名。policy-Digest 不能当全部动态权限集合的规范摘要。

43.3 接收者何时更新

User/Provider 验证并安装 Controller 状态后，版本推进会使用服务相关缓存失效；权限属性不变可以避免 DKEY 重取，但不保证业务状态毫无刷新。NDNSF_POLICY_REVALIDATION_PERIOD_MS 默认 0，未开启周期重校验时不能靠该机制主动发现。当前没有通用 SVS epoch 广播保证；认证 hint 仅触发权威拉取。目标 TG-01 将细化影响范围，不能用目标推定当前行为。

43.4 完整签名与源码定位

API-d5f6b3c470a6 public

所属符号：ndn_service_framework::ServiceController::grant

```
bool grant(const ndn::Name& identity,
           const ndn::Name& serviceName,
           const ndn::Name& authorizationAttribute);
```

源码：ndn-service-framework/ServiceController.hpp，第 103 行。

API-3ad7bdf59cc1 public

所属符号：ndn_service_framework::ServiceController::revoke

```
bool revoke(const RevocationTarget& target);
```

源码：ndn-service-framework/ServiceController.hpp，第 95 行。

API-a6e1ee327414 public

所属符号: `ndn_service_framework::ServiceController::getPolicyStatus`

```
PolicyStatusData getPolicyStatus(const ndn::Name& serviceName) const;
```

源码: `ndn-service-framework/ServiceController.hpp`, 第 91 行。

完整声明查询: `api/core-reference.md`。

44 Core API: 执行租约、观测与并发

AC-09 · Core 接口契约

44.1 租约携带什么

`GenericExecutionLease` 包含 `leaseId`、`providerName/providerEpoch`、`requesterName/requestId/serviceName/planDigest`、`resourceBindingSchema/proof`、`conflictKeys`、`expiresAtMs`、`executionDeadlineMs` 与 `idempotencyKey`。`providerEpoch` 区分 Provider 进程代次; 不能用 `ControllerVersion` 代替。`resource proof` 绑定资源, 服务权限和 DI 解密 `grant` 另行检查。

44.2 正常状态链

`prepare(lease, nowMs)` 建立 `Prepared`; `commit` 用相同租约、Provider 代次、Requester 和幂等键进入 `Committed`; `validateAndActivate` 额外验证完整 `ExecutionLeaseBinding` 与执行期限, 进入 `Executing`; 结束后 `release`。未采用的预留可 `abort`, 到期由 `cleanupExpired` 回收。`find` 返回 `optional`: 查不到与找到了 `Expired/Released` 记录需要分别处理。

44.3 结果与诊断

`ExecutionLeaseResult` 包含 `status`、`operation`、`reasonCode`、`lease`、`retryAfterMs`、`idempotentReplay`。调用者先看 `status`, 再按 `reasonCode` 处理 `LEASE_EXPIRED`、`LEASE_STALE_EPOCH`、`LEASE_IDEMPOTENCY_CONFLICT`、`LEASE_REQUESTER_MISMATCH`、`LEASE_PLAN_MISMATCH` 或 `LEASE_CAPACITY_REJECTED` 等; 重复成功回放由 `idempotentReplay` 标识。返回旧租约不等于可继续执行。

44.4 并发与观察

表内部以 `mutex` 保护状态, 不使调用它的所有 Face API 任意线程安全。`ACK/handler` 工作池各有配置, 增加线程不能扩大物理容量。`counters(nowMs)`、生命周期事件和 `diagnostics` 用于解释 `Prepared/Committed/Executing` 等资源占用; 最终完成仍需实际结果证据。

44.5 完整签名与源码定位

API-d154a2bbea78 public

所属符号: `ndn_service_framework::ProviderExecutionLeaseTable::prepare`

ExecutionLeaseResult

```
prepare(GenericExecutionLease lease, uint64_t nowMs);
```

源码: `ndn-service-framework/ExecutionLease.hpp`, 第 112 行。

API-13c34e09371a public

所属符号: `ndn_service_framework::ProviderExecutionLeaseTable::commit`

ExecutionLeaseResult

```
commit(const std::string& leaseId, const std::string& providerEpoch,  
        const std::string& requesterName, const std::string& idempotencyKey,  
        uint64_t nowMs);
```

源码: `ndn-service-framework/ExecutionLease.hpp`, 第 115 行。

API-e156fa6d6cf2 public

所属符号: `ndn_service_framework::ProviderExecutionLeaseTable::validateAndActivate`

ExecutionLeaseResult

```
validateAndActivate(const std::string& leaseId,  
                    const std::string& providerEpoch,  
                    const ExecutionLeaseBinding& binding,  
                    const std::string& idempotencyKey,  
                    uint64_t nowMs,  
                    uint64_t executionDeadlineMs);
```

源码: `ndn-service-framework/ExecutionLease.hpp`, 第 120 行。

API-f52dcb87cd76 public

所属符号: `ndn_service_framework::ProviderExecutionLeaseTable::release`

ExecutionLeaseResult

```
release(const std::string& leaseId, const std::string& providerEpoch,  
        const std::string& requesterName, const std::string& idempotencyKey,  
        uint64_t nowMs);
```

源码: `ndn-service-framework/ExecutionLease.hpp`, 第 145 行。

API-e43c54567dc6 public

所属符号: `ndn_service_framework::ProviderExecutionLeaseTable::find`

```
std::optional<GenericExecutionLease>
    find(const std::string& leaseId) const;
```

源码: `ndn-service-framework/ExecutionLease.hpp`, 第 153 行。

完整声明查询: `api/core-reference.md`。

45 Repo API: 对象与精确 Data packet

AC-10 • Repo 接口契约

45.1 对象与 manifest

`put(node, objectName, payload, options)` 的本地重载接收 `RepoNode` 和字节, 返回 `RepoObjectManifest`。StoreOptions 提供 `objectType`、`replicationFactor`、`replicaNodes`、`policyEpoch`。名字识别对象, 摘要识别内容, `segmentCount` 描述分段数量, 副本字段只是位置/策略信息, 不能用其中一个替代全部验证。网络 helper 与本地静态调用不是同一传输或超时模型。

45.2 原始 packet 例子

录像含两份已经由摄像头签名的 Data, 名称分别指向两个精确 packet。`putDataPacket` 保存原始 wire; `manifest` 保存有序完整名称; `getDataPackets` 按该列表读回。必须核对完整组的名称和索引, 不能重新签名 payload 来冒充原摄像头。普通字节对象 `put` 则不承诺这些字节本身已经加密。

45.3 存储与可见性

普通对象由 SQLite 权威存储加有界缓存支持, 权威写入后更新缓存; `RepoCore` 负责网络中立操作, `RepoNode` 持有网络服务注册。`remove` 的本地删除结果不能表示远端副本、目录或 NDN 缓存同时消失。取回 manifest 只是下一步取数依据, 完整对象成功须验证全部字节及声明摘要。

45.4 完整签名与源码定位

API-3690bad70c85 public

所属符号: `ndnsf_distributed_repo::RepoClient::put`

```
static RepoObjectManifest put(RepoNode& node,
                             const std::string& objectName,
                             const std::vector<uint8_t>& payload,
                             StoreOptions options = {});
```

源码: `NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoClient.hpp`, 第 33 行。

API-208878700957 public

所属符号: ndnsf_distributed_repo::RepoClient::getDataPackets

```
static std::vector<std::vector<uint8_t>> getDataPackets(  
    const RepoNode& node,  
    const RepoObjectManifest& manifest);
```

源码: NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoClient.hpp, 第 76 行。

完整声明查询: api/repo-reference.md。

46 Repo API: 制品上传与提交

AC-11 · Repo 接口契约

46.1 公开上传调用

publish_file 要求已有文件 path、逻辑 name、expected_sha256; replicas 默认 1、verification 默认 signed-manifest、resume 默认 True。应用必须先独立得到预期内容摘要，不能从未验证的上传结果倒填。timeout_ms、取消 token 与 progress observer 控制不同方面。

46.2 显式阶段

```
// Python 调用顺序示意  
session = api.begin_upload(descriptor)  
session.upload_file(path)  
result = session.commit()  
// transfer/upload 成功仍不能替代 commit 成功  
// 失败时按恢复策略 abort(preserve_progress=...)
```

46.3 结果字段

项目	行为与调用含义
reference / operation_id	reference 是后续下载依据，operation_id 识别本次操作；不能仅保存本地文件路径。
requested / achieved replicas	请求数量与成功数量分别报告；achieved_replicas 必须等于不同 COMMITTED receipt_id 的数量，不能只数候选 ACK。
replicas	每个 ArtifactReplicaResult 含 repo_node、state、receipt_id、error_code；用于识别哪个副本完成，哪个失败。
deduplicated / resumed	分别说明复用与恢复，不代表本轮传输了全部字节；duration 和 phase_durations_ms 是本次观测。

46.4 提交与重试

commit 校验 operation、摘要、选定副本及 receipt 后产生可交付结果。幂等键用于识别同一业务上传，但不能允许同一操作携带冲突内容。网络后端的 TARGETED 全制品发布支持有边界，不能由控制选项存在推断所有模式可用。失败保留 operation 和阶段原因；是否保留进度按 resume/abort 策略决定。

46.5 完整签名与源码定位

API-c7c484fc1ed7 public-by-name

所属符号: ArtifactRepositoryApi.publish_file

```
def publish_file(
    self,
    path: Union[str, Path],
    *,
    name: str,
    expected_sha256: str,
    replicas: int = 1,
    verification: str = "signed-manifest",
    resume: bool = True,
    on_progress: Optional[ProgressObserver] = None,
    idempotency_key: str = "",
    policy_epoch: str = "default",
    timeout_ms: Optional[int] = None,
    control: ArtifactControlOptions = ArtifactControlOptions(),
    cancellation: Optional[ArtifactCancellationToken] = None,
) -> ArtifactPublishResult:
```

源码: NDNSF-DistributedRepo/pythonWrapper/py_repodclient/artifact_api.py, 第 844 行。

API-91dcd6af85dd public-by-name

所属符号: ArtifactRepositoryApi.begin_upload

```
def begin_upload(
    self,
    descriptor: ArtifactDescriptor,
    *,
    on_progress: Optional[ProgressObserver] = None,
    cancellation: Optional[ArtifactCancellationToken] = None,
) -> ArtifactUploadSession:
```

源码: NDNSF-DistributedRepo/pythonWrapper/py_repodclient/artifact_api.py, 第 819 行。

完整声明查询: [api/repo-reference.md](#)。

47 Repo API: 下载、验证与传输状态机

AC-12 · Repo 接口契约

47.1 kwargs 的实际含义

`fetch_file(reference, destination, **kwargs)` 把参数传给 `begin_fetch`: `resume=True`、`verify=True`、`replace=False`、`on_progress=None`、`idempotency_key` 空、`timeout_ms=None`、`control=ArtifactControlOptions()`、`cancellation=None`。默认幂等键由 `reference.content_digest` 与目标绝对路径构造, `operation_identity` 还绑定 `reference`; 换路径并不天然延续同一操作。`timeout_ms` 通过 `int(timeout_ms or default_timeout_ms)` 传给后端, 所以传 0 会采用默认预算。

47.2 调用次序与结果

```
result = api.fetch_file(reference, destination,
                        resume=True, verify=True, replace=False)
// 等价主线: begin_fetch → transfer → commit
// 异常且尚未终态: 尝试 abort(preserve_progress=resume), 再抛 API error
```

47.3 如何读统计

`ArtifactFetchResult` 包含 `reference`、`operation_id`、`destination`、`reused_bytes`、`transferred_bytes`、`source_replicas`、`last_segment`、`delivered_segments`、`total_segments`、`retransmitted_bytes` 和阶段耗时。复用字节与本轮网络字节分开; `last_segment` 默认 -1, 不能当作完整对象成功。成功交付来自 `commit` 和校验, 不由进度达到某个百分比推断。

47.4 底层分段是另一层状态

`AdaptiveArtifactTransfer::receive` 记录 `segmentNo`、`logicalBytes`、`wireBytes` 和 `nowMs`, 返回 `ArtifactSegmentDisposition`; `markVerified` 显式标记验证完成。接收、验证、整体提交是不同动作。高层 `fetch session` 管理文件、操作身份和异常恢复, 底层窗口/重传状态不能直接当成最终文件完成。`resume` 必须保留同一制品和可信分段状态, 不是无条件追加目标文件。

47.5 完整签名与源码定位

API-39b8d985de2c public-by-name

所属符号: `ArtifactRepositoryApi.begin_fetch`

```
def begin_fetch(
    self,
    reference: ArtifactReference,
    destination: Union[str, Path],
    *,
    resume: bool = True,
    verify: bool = True,
    replace: bool = False,
```

```
on_progress: Optional[ProgressObserver] = None,
idempotency_key: str = "",
timeout_ms: Optional[int] = None,
control: ArtifactControlOptions = ArtifactControlOptions(),
cancellation: Optional[ArtifactCancellationToken] = None,
) -> ArtifactFetchSession:
```

源码：NDNSF-DistributedRepo/pythonWrapper/py_repodclient/artifact_api.py, 第 914 行。

API-1707faa2f3ea public-by-name

所属符号：ArtifactRepositoryApi.fetch_file

```
def fetch_file(
    self,
    reference: ArtifactReference,
    destination: Union[str, Path],
    **kwargs,
) -> ArtifactFetchResult:
```

源码：NDNSF-DistributedRepo/pythonWrapper/py_repodclient/artifact_api.py, 第 960 行。

API-6f363704fd88 public

所属符号：ndnsf_distributed_repo::AdaptiveArtifactTransfer::receive

```
ArtifactSegmentDisposition
receive(uint64_t segmentNo, uint64_t logicalBytes, uint64_t wireBytes,
        uint64_t nowMs);
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/ArtifactTransfer.hpp, 第 78 行。

API-b8a723742c4e public

所属符号：ndnsf_distributed_repo::AdaptiveArtifactTransfer::markVerified

```
void markVerified(uint64_t segmentNo);
```

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/ArtifactTransfer.hpp, 第 82 行。

完整声明查询: [api/repo-reference.md](#)。

48 Repo API: 目录、副本与缓存控制

AC-13 · Repo 接口契约

48.1 本地目录操作矩阵

项目	行为与调用含义
catalogStatus	本 Repo 的模式、catalogEpoch 和对象数；不是所有 Repo 的一致全局视图。
catalogSnapshot	返回 RepoCatalogDelta 形式的全量目录；调用方按快照恢复自己的视图。
catalogDelta(sinceEpoch)	按本 Repo 的目录 epoch 请求变化；空变化与查不到单个对象不是同一个结果，不能把其他 Repo 的 epoch 当作游标。
catalogLookup(objectName)	精确读取一个对象的 manifest，再生成 AVAILABLE 条目；无类型、类别、前缀或标签过滤参数。
Python orchestration	查询编排、目录交换和 repair 属于独立路径；C++ 类型存在不表示已在 C++ 启动这些服务。

48.2 lookup 的实际失败与锁边界

RepoCore 先在目录锁外调用 `store->get(objectName)`，再锁住目录状态构造条目和 `catalogEpoch`。对象缺失或存储异常沿路径传播，不承诺返回空集合。该顺序不保证存储与目录是一份线性一致快照；并发删除/更新时，调用方不可将条目视为永久可读凭据。`handleCatalogLookup` 将请求字节解释为对象名，返回条目 JSON。

48.3 副本判断例子

目录声称 P1、P2 保存 artifact X 时，下载还需验证 X 的 manifest 和实际字节；若 P2 不可读，不能继续统计为本轮已验证的第二副本。`repair` 应遵守 `tombstone`、`retention` 和 `repairAllowed`；旧目录 AVAILABLE 信息不能无条件复活已删除对象。TG-04 将进一步规定跨后端能力及提交/恢复一致性。

48.4 完整签名与源码定位

API-b28b415568be public

所属符号: `ndnsf_distributed_repo::RepoCore::catalogSnapshot`

`RepoCatalogDelta catalogSnapshot() const;`

源码: `NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoCore.hpp`, 第 77 行。

API-e296daab62b3 public

所属符号: `ndnsf_distributed_repo::RepoCore::catalogDelta`

```
RepoCatalogDelta catalogDelta(uint64_t sinceEpoch) const;
```

源码: `NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoCore.hpp`, 第 79 行。

API-2c18852cdee0 public

所属符号: `ndnsf_distributed_repo::RepoCore::catalogLookup`

```
RepoCatalogEntry catalogLookup(const std::string& objectName) const;
```

源码: `NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoCore.hpp`, 第 81 行。

完整声明查询: `api/repo-reference.md`。

49 DI API: Python 应用入口与原生句柄

AC-14 · DI 接口契约

49.1 Python 与 C++ 是两个现存入口

`APPClient.request` 输入 `model`、`task`、`input`、`timeout_ms`, 可带 `options/objective/constraints/strategy`。`model` 描述模型身份, `task` 指定业务任务, `input` 是应用值, 策略决定候选选择; 显式 `deployment` 的 `request` 入口属于另一种调用形态。Python `api` 包重新导出 `app_sdk`, 不意味着运行时已经委托给 C++。

49.2 原生输入与选项

`NativeApplicationInput` 区分 `Inline` 和 `RepositoryReference`, 携带任务及 `schema` 绑定; 内联字节与 `Repo` 引用不能混填。`NativeRequestOptions` 的 `timeoutMs` 默认 30000, `ackTimeoutMs` 默认 5000, 后者是前者之内的报价预算。生产路径由 `Runtime::open` → `User::prepare` → `PreparedModel` 绑定已验证模型包, 再由 `NativeInferenceClient` 创建 `handle` 并调度准备、`ACK`、`Selection`、`Provider` 与 `Response`。没有 `preparation/request contract` 的兼容构造仍明确返回 `NATIVE_REQUEST_PIPELINE_NOT_READY`; 不能把该兼容分支当作 `prepared` 路径的结果。

49.3 句柄行为

项目	行为与调用含义
<code>result(waitTimeout)</code>	局部等待超时抛 <code>LOCAL_WAIT_TIMEOUT</code> , 请求仍可继续; 不修改请求 <code>deadline</code> 。Core I/O 线程正数等待预算抛 <code>CORE_IO_WAIT_FORBIDDEN</code> 。

项目	行为与调用含义
cancel	空句柄或已终态请求不重新提交终态；返回并不保证远端计算已经停止，也不等价于释放 Provider KV。
observe	拒绝空 observer，回放已有事件；回放与新事件在独立通知队列串行排序。回调异常被捕获，捕获资源须存活至通知结束。
close / 生命周期	client close 取消当前操作并关闭调度资源；等待者持有共享 operation。迟到 dispatch 在 phase/状态不符时退出，不复活请求。
NativeDiError	保留 code/domain/boundary/requestId/attempt；调用者按错误字段分流，不能仅比较一段异常文案。

49.4 等待例子

应用给请求 30 秒预算，先用 100 毫秒局部等待显示进度。出现 LOCAL_WAIT_TIMEOUT 后可继续观察或再次等待，不应创建重复推理请求；只有应用明确结束业务才 cancel。这个例子解释句柄语义，不代表当前原生 requester 已能成功完成模型推理；接通路线见 TG-02。

49.5 完整签名与源码定位

API-0ad867060005 public-by-name

所属符号：APPClient.request

```
def request(  
    self,  
    *,  
    model,  
    task,  
    input,  
    timeout_ms: int,  
    options=None,  
    objective=None,  
    constraints=None,  
    request_id: str = "",  
    strategy=None,  
):
```

源码：NDNSF-DistributedInference/ndnsf_distributed_inference/app_sdk/client.py, 第 1096 行。

API-375b92c9a402 public

所属符号: ndnsf::di::NativeInferenceClient::request

```
NativeInferenceHandle request(  
    const NativeModelRef& model,  
    const NativeApplicationInput& input,  
    std::shared_ptr<const NativeModelSplitStrategy> splitStrategy,  
    std::shared_ptr<const NativePlacementStrategy> placementStrategy,  
    const NativeRequestOptions& options);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceClient.hpp, 第 219 行。

API-c0c25988d534 public

所属符号: ndnsf::di::NativeInferenceHandle::result

```
NativeInferenceResult result(std::chrono::milliseconds waitTimeout) const;
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceClient.hpp, 第 157 行。

完整声明查询: [api/di-reference.md](#)。

50 DI API: 模型适配与角色放置扩展

AC-15 · DI 接口契约

50.1 规划对象怎么连接

项目	行为与调用含义
模型描述 / 图快照	描述模型及 adapter 身份;inspect 得到真实图,planning graph 与 canonical ONNX graph 可有不同摘要,不能只因都叫 graphDigest 就混用。
NativeSplitCandidate	enumerate 返回的候选包含模型/图绑定和角色方案;候选数量受预算控制。候选是可验证的执行分解,不是任意 Provider 名单。
ProviderOfferV3	来自认证 ACK 的报价,经 admission 检查请求、attempt、模型/图、Provider、设备和新鲜度;不是从 capability 文本自行补全的运行承诺。
V3 proposal	proposeRoles 以绑定上下文、ackClosedDigest、角色、合格报价和 nowMs 提出放置;独立 validator 仍要校验覆盖、资源和身份。

50.2 两段模型例子

将线性模型的前半段作为 A、后半段作为 B,候选须说明 A 的输出与 B 的输入匹配。P1 报价能运行 A, P2 报价能运行 B, 策略才可提出 $A \rightarrow P1$ 、 $B \rightarrow P2$; 如果 P2 只有模糊 has_model 声明而缺该角色/设

备证明，不能标为 exact reuse。若允许准备，则明确采用需装配/加载的路径，并把准备成本计入总期限。

50.3 扩展最小职责

NativeModelSplitStrategy 实现 identity 和 enumerate；NativePlacementStrategy 实现 identity 和 proposeRoles。adapter 拥有模型检查、输入编码、结果解码，由 registry 登记并在使用前固定。策略返回值不能给自己签发 grant。新增策略应以固定图、报价、时钟测试确定性和拒绝分支，再让实际 requester 选择它；仅创建子类不会改变默认路径。

50.4 当前源码的描述符与候选增补

当前工作树 NativeModelDescriptor 包含 NativeAdapterDescriptor 和 sourceRevision；adapter 描述符绑定 name/version/stateDigest/abi、格式/任务/backend/precision 能力及输入/options/result schema 摘要，canonicalJson/modelDigest 提供规范身份。NativeRequestPreparation 的 sameModel 按模型 canonicalJson 比较。当前 NativeSplitCandidate 还包含 fragmentsByRole、artifactsByRole、requirementsByRole、crossPartitionTensors、tensorDegreesByRole、rankArtifactDigestsByRole、输入/结果角色、mergeKind、postprocessingJson、estimatedCosts、可选 hybridPlan 与 candidateDigest；computedDigest 用于完整候选身份，不能只哈希部分角色名。这些是并行实现中的源码事实，完整语义、独立 oracle 与接线验收仍以 Spec182 为准；目标冻结声明不自动复制这些新增字段。

50.5 完整签名与源码定位

API-da75fc521e8b public

所属符号：ndnsf::di::NativeModelSplitStrategy::enumerate

```
virtual std::vector<NativeSplitCandidate> enumerate(
    const NativeModelDescriptor& model,
    const NativeGraphSnapshot& graph,
    const NativeCandidateBudget& budget) const = 0;
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanning.hpp，第 301 行。

API-0f4ac0e590fd public

所属符号：ndnsf::di::NativePlacementStrategy::proposeRoles

```
virtual NativeRolePlacementProposalV3 proposeRoles(
    const NativeOfferBindingContext& context, const std::string& ackClosedDigest,
    const std::vector<NativeSelectionRoleV3>& roles,
    const std::vector<NativeAdmittedOfferV3>& offers, std::uint64_t nowMs) const = 0;
```

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanning.hpp，第 312 行。

完整声明查询：api/di-reference.md。

51 DI API：准备、发布后再绑定与封存

AC-16 · DI 接口契约

51.1 阶段输入输出

项目	行为与调用含义
prepareInput / inspect-Model	绑定 task/schema、输入字节或引用和 NativeRequestControl；检查真实模型与图。control 保存 request/attempt、steady-clock deadline 和取消检查。
prepareRoles / validateRoles	把候选转成可执行角色并检查一致性，不能先封存再猜测模型边界。
ensureArtifacts	消费完整 candidate、proposal、control，通过发布端口确保制品；检查业务 root 原始字节及模型/profile/source/package 等身份。
bindPublishedRoles	将发布后的 manifest/recipe 绑定到角色；保持角色、候选、Provider、设备选择不被偷偷更换，旧 exact-reuse 证明必须仍覆盖实际 recipe。
sealCore / grantView	前者固定计划核心，后者形成授权所需视图；摘要对应具体规范对象，不是任意 JSON 文本。
finalizeSecurity / project / encode	完成安全材料后产生各 Provider 的选择投影并编码；不能在安全材料未绑定时发送一个看似完整的旧投影。

51.2 发布改变身份的例子

候选最初引用 canonical 模型，发布后得到切片 artifact root R 和装配 recipe Q。不能只把 URL 改为 R 而继续保留旧 planDigest 或驻留证明；bindPublishedRoles 校验选择不变并绑定 R/Q，再封存对应计划与授权。业务 root 摘要标识模型制品，Core 传输 manifest 描述取数封装，两者不能互代。

51.3 失败边界

取消/过期先由 control 拒绝继续准备；内容或角色不匹配在准备/再绑定阶段拒绝，grant 验证失败在安全边界拒绝。端口定向测试不证明生产 Repo/catalog 与 requester 已接通；实现状态集中见第 31 章与 Spec182，而非给每一步追加成功承诺。

51.4 完整签名与源码定位

API-f8cbacddff79 public

所属符号： ndnsf::di::NativeRequestPreparation::prepareInput

```
NativePreparedInput prepareInput(const NativeModelDescriptor& model,
                                std::string taskName,
                                std::string inputSchemaDigest,
                                std::string optionsSchemaDigest,
                                std::vector<std::uint8_t> payload,
                                std::string repositoryReference,
                                std::chrono::steady_clock::time_point deadline) const;
```

源码： NDNSF-DistributedInference/cpp/ndnsf-di/NativeRequestPreparation.hpp, 第 108 行。

API-dcca376d355a public

所属符号： ndnsf::di::NativeRequestPreparation::inspectModel

```
NativeInspectedModel inspectModel(const NativePreparedInput& input) const;
```

源码： NDNSF-DistributedInference/cpp/ndnsf-di/NativeRequestPreparation.hpp, 第 116 行。

API-2d8e39c7fc67 public

所属符号： ndnsf::di::NativeRequestPreparation::ensureArtifacts

```
NativeArtifactBinding ensureArtifacts(const NativeInspectedModel& model,
                                      const NativeSplitCandidate& candidate,
                                      const NativeRolePlacementProposalV3& proposal,
                                      const NativeRequestControl& control) const;
```

源码： NDNSF-DistributedInference/cpp/ndnsf-di/NativeRequestPreparation.hpp, 第 124 行。

API-d192a8a670da public

所属符号： ndnsf::di::NativeRequestPreparation::bindPublishedRoles

```
static std::vector<NativeSelectionRoleV3> bindPublishedRoles(
    const NativeInspectedModel& model, const NativeSplitCandidate& candidate,
    const std::vector<NativeSelectionRoleV3>& roles, const NativeArtifactBinding& artifacts);
```

源码： NDNSF-DistributedInference/cpp/ndnsf-di/NativeRequestPreparation.hpp, 第 131 行。

API-1df2153e06a6 public

所属符号: `ndnsf::di::NativePlanSealer::sealCore`

```
static NativePlacementPlanCore sealCore(  
    const NativePlanningSnapshot& snapshot,  
    const NativePlacementProposal& proposal,  
    const NativePlanSealingInputs& inputs);
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanSealer.hpp`, 第 117 行。

API-f5591fadba7d public

所属符号: `ndnsf::di::NativePlanSealer::grantView`

```
static NativeProviderGrantView grantView(  
    const NativePlacementPlanCore& core,  
    const NativeProviderPlanningView& provider,  
    const NativeSecurityPolicySnapshot& security,  
    const std::string& role = {});
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanSealer.hpp`, 第 128 行。

API-e3e459f19b03 public

所属符号: `ndnsf::di::NativePlanSealer::project`

```
static NativeSelectionProjectionV3 project(  
    const NativeSealedPlan& sealed, const std::string& provider,  
    const NativeRoleProjectionInputs& inputs);
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanSealer.hpp`, 第 143 行。

API-07a74268daf4 public

所属符号: `ndnsf::di::NativePlanSealer::encode`

```
static std::vector<std::uint8_t> encode(  
    const NativeSelectionProjectionV3& projection);
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativePlanSealer.hpp`, 第 147 行。

完整声明查询: `api/di-reference.md`。

52 DI API: Provider 注册、授权与执行

AC-17 · DI 接口契约

52.1 组成一个 Provider

NativeServiceDefinition 声明 serviceName 和模型支持, NativeProviderHandlerConfig 提供处理配置; serve 返回必须保存的 NativeServiceRegistration。执行层 NativeProviderSession 持有 plan、assignment、DependencyIo、runnerFactory 和 worker 数, 按角色 registerRunner。plan 描述做什么, assignment 指定谁做, DependencyIo 指定怎样获取/发布边数据, runnerFactory 提供实际后端。

52.2 执行次序

Core scoped handler 接受并验证 Selection 后, NativeProviderHandler 取得协作认证上下文, 核对计划、角色、lease 和 grant; 再构造/调用 session.executeRoleAsync 返回 future<ProviderRoleResult>, 调用方须处理 future 的异常并保留 session/I/O 生命周期。source role 可用初始输入, 有依赖的角色等直接前驱输出, 不需要凭空增加全局所有节点 Ready 屏障。

52.3 grant 从哪里来

verifyAndUnwrapNativeGrant 的输入来自认证选择上下文和 Provider 投影, 不能使用应用随意填写的 requester/plan 字符串代替已认证来源。服务权限允许调用, execution lease 约束资源, request-scoped grant 约束受保护模型与计划; 三个条件不能互相代替。默认 plaintext-v1 路径与 ProtectedRuntime 明文生命周期管理要分别说明。

52.4 关闭与迟到工作

保存服务注册句柄直到停服, close 阻断该 generation 的新工作和晚到发布; 应用还需排空已启动 worker。关闭服务、释放运行 lease、释放缓存状态和删除 artifact 是不同操作。真实后端/设备以及迁移接线状态由第 31 章与当前 Spec 记录。

52.5 完整签名与源码定位

API-b5abfc9b9043 public

所属符号: ndnsf::di::NativeInferenceProvider::serve

```
NativeServiceRegistration serve(const NativeServiceDefinition& service,
                               const NativeProviderHandlerConfig& config);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceProvider.hpp, 第 103 行。

API-1ec58fcbe7cc public

所属符号: `ndnsf::di::NativeProviderSession::NativeProviderSession`

```
NativeProviderSession(NativeExecutionPlan plan,
                      NativeProviderAssignment assignment,
                      std::shared_ptr<DependencyIo> dependencyIo,
                      std::shared_ptr<NativeModelRunnerFactory> runnerFactory,
                      std::size_t workerCount = std::thread::hardware_concurrency());
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp`, 第 19 行。

API-79b6adfd9bf2 public

所属符号: `ndnsf::di::NativeProviderSession::registerRunner`

```
void
registerRunner(const NativeModelRunnerSpec& spec);
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp`, 第 25 行。

API-5ea3dea813e9 public

所属符号: `ndnsf::di::NativeProviderSession::executeRoleAsync`

```
std::future<ProviderRoleResult>
executeRoleAsync(const std::string& sessionId, const std::string& role);
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp`, 第 34 行。

API-070f116bfa58 public

所属符号: `ndnsf::di::verifyAndUnwrapNativeGrant`

```
NativeGrantVerificationResult
verifyAndUnwrapNativeGrant(const std::string& wireJson,
                           const std::string& authorityPublicKeyRaw,
                           const NativeRecipientKey& recipientKey,
                           const std::string& providerIdentity,
                           const std::string& requestId,
                           std::uint64_t attempt,
                           const std::string& planCoreDigest,
                           const std::string& modelManifestDigest,
                           const std::string& protectionEpoch,
                           std::uint64_t nowMs,
                           const std::string& expectedAuthority = "",
                           const std::string& expectedGrantDigest = "");
```

源码: `NDNSF-DistributedInference/cpp/ndnsf-di/NativeGrantVerifier.hpp`, 第 57 行。

完整声明查询: [api/di-reference.md](#)。

53 DI API：生成、KV 与会话延续

AC-18 · DI 接口契约

53.1 四个 owner 与真正执行入口

实际生成入口是自由函数 `runNativeEpochCoordinator(config)`，不是一个名为 `NativeEpochCoordinator` 的对象方法。它组织各 `role/epoch` 执行；`QwenGenerationSessionStateMachine` 管理合法动作和终止原因；`NativeProviderSession/Runtime` 管理 `Provider` 解码/会话缓存；`NativeConversationCoordinator` 管理跨轮 `checkpoint` 的本地记录。请求句柄只负责外层等待/取消/观察，不承担这些全部职责。

53.2 Config 与结果中的关键字段

项目	行为与调用含义
<code>runtime / plan / assignment / dependencyIo</code>	构造 <code>config</code> 时提供： <code>runtime</code> 是借用对象， <code>plan/assignment</code> 随 <code>config</code> 保存， <code>I/O</code> 为共享依赖。运行期间这些资源必须有效。
<code>sessionId / requestId / role</code>	分别识别会话、请求和本地执行角色， <code>localProvider</code> 绑定本地参与者。 <code>lineagePlanDigest</code> 来自认证的计划投影。
<code>attemptEpoch / streamEpoch</code>	默认均为 1； <code>attempt</code> 表示替换尝试， <code>stream</code> 表示流实例，不是每输出一个 <code>token</code> 都增加 <code>streamEpoch</code> 。
<code>initialInputs / tokenInputName</code>	初始张量按名字映射， <code>tokenInputName</code> 默认 <code>input_ids</code> ； <code>stateInputNames/stateOutputNames</code> 描述模型状态端口。
<code>maxEpochs / sampling</code>	<code>maxEpochs</code> 默认 0，调用方需按有效配置设置界限； <code>samplingMode</code> 默认 <code>Greedy</code> ， <code>temperature=0</code> 、 <code>topK=1</code> 、 <code>topP=1</code> 、 <code>repetitionPenalty=1</code> ， <code>seed=1750001</code> 。不由默认值推断随机采样已验收。
<code>stateIdentityTemplate</code>	由 <code>adapter/runner</code> 提供不可变状态身份模板，运行时派生前缀/计数相关身份； <code>conversationStateBinding</code> 指向父轮保留状态，两者不是同一对象。
<code>checkStop / 文本解码</code>	<code>checkStop</code> 在工作边界报告 <code>Cancelled/Deadline</code> ； <code>stableTextDecoder(tokens, final)</code> 可暂留不稳定后缀， <code>final</code> 时完成 <code>flush</code> 。
结果	<code>finalPayload</code> 为 <code>optional</code> ； <code>epochsExecuted/eventsPublished/prefixTokensRecomputed</code> 是实际计数。 <code>finalizedRole</code> 是最后提交的准确角色身份，用于会话状态提升，不应由应用重新拼装。

53.3 Qwen 状态机：允许的调用

构造函数先 `validate session spec`；`spec` 绑定 `candidate/plan/model/artifact`、`logicalSessionId/requestId/serviceName`、`attempt/token epoch`、输入与生成 `token` 上界、`deadline`、`context/feedback` 与角色。下表描述实际方法分支。`Preparing` 虽在枚举中存在，但 `activate` 在此实现直接进入 `Prefilling`，

不能据枚举名称补造一条执行链。

项目	行为与调用含义
Created → Selecting	beginSelection, 仅 Created 可调用。
Selecting/Rebuilding → Prefilling	activate; 其他状态抛 logic_error。
Prefilling → Decoding	completePrefill; 之后才允许 token 计数和停止条件观察。
Decoding → Decoding	completeTokenEpoch(attempt) 要求 attempt 相同且未达 maxGenerated-Tokens, 返回增加后的生成计数; 旧 attempt 或越界抛 logic_error。
Decoding → Draining	beginDrain(reason): Eos/StopSequence 先有对应 observe; MaxTokens 要计数恰好达到上限; ApplicationComplete 无这两项观察要求。
Draining → Completed	complete 必须使用相同 finish reason。claimTerminalResponse 第一次返回 true, 后续 false; 终态前调用抛异常。它只领取发送权, 不负责网络发布。
Decoding → Rebuilding	beginReplacement 增加 attempt, 最多到 2; 再替换抛异常。生成计数保留, 不把已交付前缀清零。
取消 / 期限 / 失败	cancel 进入 Cancelled, 已终态再次调用抛 logic_error; terminate 要求非终态且 reason 非 None; expireIfDeadlineReached 在尚未到期或已终态时返回 false。

53.4 两 token 示例与替换

```
// 示意: validSpec 已通过模型/角色/摘要与 deadline 校验
QwenGenerationSessionStateMachine state(validSpec);
state.beginSelection();
state.activate();
state.completePrefill();
auto n1 = state.completeTokenEpoch(state.attemptEpoch());
auto n2 = state.completeTokenEpoch(state.attemptEpoch());
// 若生成上限是 2, 按 MaxTokens 进入 Draining, 再以同一原因 complete
// 终态发布方必须先 claimTerminalResponse(), 仅成功领取者发布
```

53.5 token 循环与重复抑制

epoch coordinator 每角色/轮次调用 NativeModelRunner::run; 此路径不使用兼容的单 Provider run-Streamed。attempt 2 的 committedPrefixTokenIds 用于重算并核对已提交前缀, 抑制重复事件。缓存观测中的 actualNewInputExtent 可能是 token 数, 也可能是激活元素数量, 不能一律计为 token。checkpointFinalize 是保留会话时的状态补齐阶段, 默认 false、上限 32; 不发布新的 token 事件。终端文本、事件数量与 KV 包含的前缀必须分别对齐。

53.6 KV 操作与资源状态

项目	行为与调用含义
lookupConversationState	返回 optional<TensorBundle>; 空结果不能作为命中。binding 校验错误可能抛异常, 不是所有失败都返回空值。
pin / unpin	pin 校验 binding、状态与有效期并增加 pinCount; unpin 递减, 计数归零回到 IDLE。pin 不是续期或重新授权。
pauseConversationStateToHost	仅匹配、未过期、GPU_RESIDENT、IDLE 且未 pin 的条目可迁移; Host 空间不足或 adapter 暂停失败返回 false。它改变驻留位置, 不新建对话轮次。
prefetchConversationStateToGpu	返回 future<bool>, 必须等待结果并处理异常; 发起预取不表示已在 GPU。cancelConversationStatePrefetch 与请求取消是不同动作。
releaseConversationState	按 binding 释放缓存条目; 与 unpin (减少引用保护) 不同, 也不删除已经保存的会话 journal。
提升缓存	promoteDecodeStateToConversation 使用真实执行后的 session/role 和 binding; 不可只凭请求号复用另一模型、recipe、前缀或安全域的 KV。

53.7 从第 N 轮延续到 N+1 轮

前提是 coordinator 已有父轮记录 (或先 restore 加载), Provider 仍有与父 binding 匹配的状态。beginTurn 不能单独启动一个从未登记的新对话: 它要求非空 conversationId/serviceName/planRoleMapDigest/parentCheckpointDigest/requestContractDigest、正 parentContextEpoch/retentionDeadlineMs/request attempt, 并在内存记录中核对父 epoch、摘要、服务和角色映射; 不匹配抛 DI_NATIVE_CONVERSATION_PARENT_MISMATCH。这里仅检查 retention deadline 非零, 不自行完成墙钟过期验证。

53.8 checkpoint 的三个步骤

```
// continuation 指向已登记父记录; requestId 为本轮新请求
auto turn = journal.beginTurn(continuation, requestId, 1);
// 真实生成结束后填写 completed: 同 request/attempt、tokenIds、
// providerStateDigest, 且 complete=true; 不能用局部 token 事件冒充
auto checkpoint = journal.prepareCheckpoint(turn, completed);
auto record = journal.commitTurn(turn, checkpoint);
// 下一轮引用 record.checkpoint 的 successor epoch 与摘要
```

53.9 journal 的实际限制

prepareCheckpoint 拒绝 aborted、request/attempt 不同、未完成、空 token 列表或空 providerStateDigest, 报 DI_NATIVE_CONVERSATION_ATTEMPT_INCOMPLETE。prefixDigest 当前由包含会话/轮次/请求/父摘要/Provider 状态及 token 列表的拼接材料计算, 不是仅 token 原始字节摘要。commitTurn 检查父记录未变化, 否则报 DI_NATIVE_CONVERSATION_STATE_CHANGED; 写临时文件再 rename 并更新内存。当前没有显式 fsync/输出流失败检查, 不能宣称崩溃耐久提交。restore 清空并读取本地 JSON, 解析错误可能传播; 它不会恢复远端 GPU KV。abortTurn 当前没有持久化或实际修改 turn.aborted 的动

作，不能把注释中的 fence 当已实现取消屏障。上述缺口保留为目标改进和 Spec182 待验收边界。

53.10 完整签名与源码定位

API-d18cce8f82c2 public

所属符号: ndnsf::di::runNativeEpochCoordinator

```
NativeEpochCoordinatorResult  
runNativeEpochCoordinator(NativeEpochCoordinatorConfig config);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeEpochCoordinator.hpp, 第 163 行。

API-493dc84e5029 public

所属符号: ndnsf::di::QwenGenerationSessionStateMachine::completeTokenEpoch

```
std::uint32_t completeTokenEpoch();
```

源码: NDNSF-DistributedInference/cpp/adapters/qwen/QwenGenerationSession.hpp, 第 146 行。

API-5772f7ae757f public

所属符号: ndnsf::di::QwenGenerationSessionStateMachine::completeTokenEpoch

```
std::uint32_t completeTokenEpoch(std::uint64_t attemptEpoch);
```

源码: NDNSF-DistributedInference/cpp/adapters/qwen/QwenGenerationSession.hpp, 第 147 行。

API-6fd4334a3745 public

所属符号: ndnsf::di::QwenGenerationSessionStateMachine::beginReplacement

```
void beginReplacement();
```

源码: NDNSF-DistributedInference/cpp/adapters/qwen/QwenGenerationSession.hpp, 第 151 行。

API-30fbe53f3d3e public

所属符号: ndnsf::di::QwenGenerationSessionStateMachine::cancel

```
void cancel();
```

源码: NDNSF-DistributedInference/cpp/adapters/qwen/QwenGenerationSession.hpp, 第 154 行。

API-855d3ec7afd2 public

所属符号: ndnsf::di::QwenGenerationSessionStateMachine::claimTerminalResponse

```
bool claimTerminalResponse();
```

源码: NDNSF-DistributedInference/cpp/adapters/qwen/QwenGenerationSession.hpp, 第 156 行。

API-1013f1ace7ee public

所属符号: ndnsf::di::NativeProviderSession::lookupConversationState

```
std::optional<TensorBundle>  
lookupConversationState(const ConversationStateBinding& binding,  
                        std::uint64_t nowMs);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp, 第 51 行。

API-10751ce7a09d public

所属符号: ndnsf::di::NativeProviderSession::pauseConversationStateToHost

```
bool  
pauseConversationStateToHost(const ConversationStateBinding& binding,  
                             std::uint64_t nowMs);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp, 第 55 行。

API-6b3bff150575 public

所属符号: ndnsf::di::NativeProviderSession::prefetchConversationStateToGpu

```
std::future<bool>  
prefetchConversationStateToGpu(const ConversationStateBinding& binding,  
                               std::uint64_t nowMs);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp, 第 59 行。

API-daca2274c659 public

所属符号: ndnsf::di::NativeProviderSession::pinConversationState

```
bool  
pinConversationState(const ConversationStateBinding& binding,  
                    std::uint64_t nowMs);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp, 第 66 行。

API-32c8162259ef public

所属符号: ndnsf::di::NativeProviderSession::releaseConversationState

bool

```
releaseConversationState(const ConversationStateBinding& binding);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeProviderSession.hpp, 第 73 行。

API-1e7281b52fd5 public

所属符号: ndnsf::di::NativeConversationCoordinator::beginTurn

```
NativeConversationTurn beginTurn(const NativeConversationContinuation& continuation,  
                                std::string requestId,  
                                std::uint64_t attempt = 1) const;
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeConversationCoordinator.hpp, 第 105 行。

API-a550281a7724 public

所属符号: ndnsf::di::NativeConversationCoordinator::prepareCheckpoint

```
NativeConversationCheckpoint prepareCheckpoint(  
    const NativeConversationTurn& turn, const NativeCompletedAttempt& completed) const;
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeConversationCoordinator.hpp, 第 126 行。

API-8c7c62356113 public

所属符号: ndnsf::di::NativeConversationCoordinator::commitTurn

```
NativeConversationRecord commitTurn(const NativeConversationTurn& turn,  
                                    const NativeConversationCheckpoint& checkpoint);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeConversationCoordinator.hpp, 第 128 行。

API-64cbde6b1261 public

所属符号: ndnsf::di::NativeConversationCoordinator::restore

```
void restore(const std::filesystem::path& journalRoot);
```

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeConversationCoordinator.hpp, 第 130 行。

API-7014d4b50f1d public

所属符号: ndnsf::di::NativeConversationCoordinator::restore

void restore();

源码: NDNSF-DistributedInference/cpp/ndnsf-di/NativeConversationCoordinator.hpp, 第 131 行。

完整声明查询: api/di-reference.md。

54 UAV API: 飞控边界与操作权限

AC-19 · UAV 接口契约

54.1 内部接口与线载荷

FlightControllerBackend 是 Drone 应用内部接口, sendMavlink(frame, commandName) 接收原始 MAVLink 字节与命令标签, 返回 Fields 键值结果。Fields 不提供统一静态类型/单位检查; 调用者必须遵循 UavProtocol 与当前服务 handler 的命令字段解析, 不能把任意 map 当已经过认证的命令。AC-19 所列为接口声明, Mock/UDP/串口行为要按实际 backend 分开。

54.2 完整命令路径

GS 从 UI 状态和操作选择构造命令, 经 /UAV/MAVLink/Execute 调用目标 Drone; Core 检查服务权限、token 和认证绑定, UAV 路径处理操作 lease, 应用/飞控处理 readiness 与执行。收到网络 Response、backend 发送字节和飞控 COMMAND_ACK 是三个事件; UI 应保留失败所处阶段, 不能把一次本地 send 成功显示成车辆已经执行。

54.3 三类状态的例子

项目	行为与调用含义
Core 授权失败	尚未获得服务执行资格; 修复身份/策略, 不能重新解释为飞控忙。
OperatorAuthorityLease 失效	操作员对目标/范围/期限的应用授权不成立; issuer 不是 ServiceController, lease 版本不是 Controller epoch。
readiness 不满足	例如应用所需新鲜遥测缺失; lease 有效不意味着所有飞行动作都可执行。
Mock 与设备 backend	Mock 返回可用于界面/状态机检查的结果; UDP/串口涉及线程、链路、ACK 和设备错误, 其成功不能从 Mock 外推。

54.4 扩展边界

新 backend 实现能力、遥测、命令结果与停止行为，保留实际错误。GS Qt 回调不能延长已销毁设备对象的生命周期。当前仍缺一份覆盖所有命令的规范字段/ACK 表，不能把本章当完整飞控线协议规范；TG-05 规定类型化与兼容转换的收敛方向。

54.5 当前 Execute 字段与 backend 返回

Fields 的实际类型是 `map<string,string>`。Drone Execute handler 读取 `command`（缺省 `unknown`）和 `mavlink_hex`（缺省空字符串），`hexDecode` 后直接调用 `backend->sendMavlink`；返回附加 `backend` 和 `drone_id`，以 `accepted` 字符串是否为 `true` 决定外层响应成功。此 handler 内未见 `OperatorAuthorityLease` 或 `readiness` 的独立重验；GS 的操作门与 UI 拒绝不能作为该网络入口已强制执行这两项检查的证据。

项目	行为与调用含义
<code>accepted / ack_result</code>	Mock 返回 <code>mock-accepted/mock-rejected</code> 。设备 backend 连接/发送失败分别返回 <code>connect-failed/send-failed</code> ；其他命令以 <code>accepted</code> 或 <code>in-progress</code> ACK 判为 <code>accepted=true</code> 。
<code>manual_control</code>	成功发送后仅收集约 5 ms 遥测，设置 <code>manual-control-forwarded</code> ，并据转发判为接受，不等同等待 <code>COMMAND_ACK</code> 。
其他设备命令	<code>sendMavlink</code> 传入 700 ms ACK 等待预算；整次调用还含连接和发送，不保证总耗时严格等于 700 ms。内部持 <code>socket mutex</code> ，可能阻塞调用线程。
<code>forwarded_bytes</code>	记录帧大小； <code>send-failed</code> 分支也填 <code>frame.size()</code> ，不是实际成功发送字节的可靠统计。
<code>ack_source / fc_state</code>	<code>ack_source</code> 是 mock 或实际 <code>transport</code> ； <code>fc_state</code> 及附加遥测由后端产生，字段集合并非所有 backend 完全一致。

54.6 完整签名与源码定位

API-b1e46a3def66 public

所属符号：FlightControllerBackend::sendMavlink

```
virtual Fields sendMavlink(const std::vector<uint8_t>& frame,
                          const std::string& commandName) = 0;
```

源码：NDNSF-UAV-APP/drone/DroneServiceContainer.inc.hpp，第 8 行。

完整声明查询：api/uav-reference.md。

55 UAV API: 任务、补偿与终端报告

AC-20 · UAV 接口契约

55.1 任务和调用各自存活多久

UavMissionSession 持有长生命周期 mission; part 是可独立完成的工作单元, incident/job 是有限分析调用。assignPart、markPartExecuting、completePart、markPartMissing 维护 part。Core requestId/token 属于每次 invocation, 新 attempt 不沿用旧调用令牌。

55.2 两个 part 的补偿例子

任务含 P0、P1: P0 已完成并有有效结果, P1 失联进入 missing。compensateMissingParts 只为 P1 形成补偿工作, 不重跑 P0。补偿使用新的 attempt/调用绑定; 若旧 P1 结果晚到, acceptTerminalReport 仍要匹配当前任务、job、attempt、计划与 terminal owner, 不能覆盖新 attempt。全部必要 part 有有效结果才是 mission 完成。

55.3 返回、恢复与资源

bool 表示本地迁移/报告接受, reason 指针可接收诊断; false 不表示车辆已停止。snapshot/restore 保存/载入业务状态, reconcileVehicleAndStreams 再与实际车辆/流对照; 不能恢复一个 Active 标记就继续发命令。incident 的 retryAnalysis/retriesUsed 另有有界重试, 不由 mission 无限重试代替。

55.4 完整签名与源码定位

API-9439cf6844b2 public

所属符号: ndnsf::examples::uav::UavMissionSession::compensateMissingParts

```
bool compensateMissingParts(std::string* reason = nullptr);
```

源码: NDNSF-UAV-APP/shared/UavMissionSession.hpp, 第 49 行。

API-8bc23c85a02b public

所属符号: ndnsf::examples::uav::UavMissionSession::acceptTerminalReport

```
bool acceptTerminalReport(const UavTerminalReport& report,  
                          std::string* reason = nullptr);
```

源码: NDNSF-UAV-APP/shared/UavMissionSession.hpp, 第 60 行。

API-f10389b9d21d public

所属符号：ndnsf::examples::uav::UavIncidentCoordinator::acceptReport

```
bool acceptReport(const UavTerminalReport& report, uint64_t nowMs,
                  std::string* reason = nullptr);
```

源码：NDNSF-UAV-APP/ground-station/UavIncidentCoordinator.hpp，第 62 行。

完整声明查询：api/uav-reference.md。

56 UAV API：视频管线、遥测与录像

AC-21 · UAV 接口契约

56.1 队列输入与准确返回

BoundedLatestFrameQueue 构造要求 maxFrames 和 maxBytes 都为正,否则 invalid_argument.push 接收含字节的 UavVideoFrame；单帧超过字节容量返回 false 并记录 frame-exceeds-byte-capacity；否则先丢最旧帧直到数量和字节均容纳新帧，再入队返回 true。因此 true 也可能伴随历史帧被丢弃。

56.2 popLatest 的例子

队列依次存有帧 10、11、12，popLatest 只返回 12，把 10、11 计为 dropped，并清空队列和字节计数；空队列返回 nullopt。此策略适合及时显示，不能用于要求逐段完整的制品下载或完整录像。push/pop/clear 的内部 mutex 保护队列，不自动保护外部解码器。

56.3 遥测值与时间

项目	行为与调用含义
地理/速度/电量	latitudeE7/longitudeE7 为角度的 10^7 缩放，altitudeMm 为毫米，groundSpeedMmps 为毫米每秒，batteryPermille 为千分比。
sampleId / sourceTimestampNs	分别用于样本身份和源时间；receivedTimestampNs 是接收时间，不能与源时间混作同一观测。
admit 返回	valid/stateAdvanced/newSample/duplicate/outOfOrder/ageNs/reason 分开解释；格式有效的旧样本不一定推进最新状态。
声学块	CompleteAcousticBlockAdmission 要求领域完整性和截止期；Core 的 FEC/签名恢复不直接等价于声学业务可用。

56.4 捕获到录像的分工

capture/编码产生带 session、frameId、PTS、keyframe 等元数据的帧, Core 流负责命名传输, GS 解码后经最新帧队列显示。录像路径保存原签名/保护 packet 与应用 manifest 到 Repo, 记录 gap、retention 与游标; 直播显示丢帧和录像数据缺口分别统计。相机链已有具体入口, 不能据 DataProductReference 推定所有遥测/检测结果已自动录制。

56.5 完整签名与源码定位

API-a064d74ab888 public

所属符号: ndnsf::examples::uav::BoundedLatestFrameQueue::push

```
bool push(UavVideoFrame frame);
```

源码: NDNSF-UAV-APP/shared/UavVideoPipeline.hpp, 第 65 行。

API-11ef609f2477 public

所属符号: ndnsf::examples::uav::BoundedLatestFrameQueue::popLatest

```
std::optional<UavVideoFrame> popLatest();
```

源码: NDNSF-UAV-APP/shared/UavVideoPipeline.hpp, 第 66 行。

API-730501b40b2c public

所属符号: ndnsf::examples::uav::LatestTelemetryAdmission::admit

```
TelemetryAdmissionResult admit(const std::vector<uint8_t>& wire,
                                uint64_t receivedTimestampNs);
```

源码: NDNSF-UAV-APP/shared/UavSensorStreams.hpp, 第 57 行。

完整声明查询: [api/uav-reference.md](#)。

57 UAV API: 检测与多视角识别扩展

AC-22 · UAV 接口契约

57.1 输入对象与责任

MultiViewRecognitionJob 绑定任务/attempt、目标与精确视图引用; model profile 指定模型输入要求, verified views 提供已验证视图, terminalOwner 指定结果归属。executeMultiViewJob 返回 MultiViewExecutionResult; 入口先通过 validateMultiViewJobAndViews 校验任务、视角数量/来源、时间窗口和模型绑定, 不能让算法自行忽略缺视角。

57.2 两个视角的例子

Drone A 和 B 在同一事件窗口提供两份已验证画面引用, job 指向这两份精确对象。选定算法消费对应视图并产生结果, `UavIncidentCoordinator::acceptMultiViewResult` 再校验当前 job/attempt/owner。把 A 的同一画面重复两次不应作为两架独立 Drone 的证据; 单视角基线另行构造输入。

57.3 算法扩展与错误

`IMultiViewRecognitionAlgorithm` 是 C++ 扩展点; 默认 `DeterministicMultiViewAlgorithm` 是确定性研究/测试夹具。真实 ONNX CPUExecutionProvider 路径在 `tools/mvcnn_onnx_worker.py`, 必须显式核对调用方接线。缺视角、模型/摘要不符、任务过期、错误 owner 与模型执行失败保留各自原因。GS 的单帧 YOLO、UAV 多视角 job、NDNSF-DI 规划是三个入口, 不能混称一次分布式推理。

57.4 完整签名与源码定位

API-1da9ec7348c6 public

所属符号: `ndnsf::examples::uav::executeMultiViewJob`

```
MultiViewExecutionResult
executeMultiViewJob(const MultiViewRecognitionJob& job,
                    const MultiViewModelProfile& profile,
                    const std::vector<VerifiedMultiViewInput>& views,
                    const ndn::Name& terminalOwner,
                    const IMultiViewRecognitionAlgorithm& algorithm =
                        DeterministicMultiViewAlgorithm{});
```

源码: NDNSF-UAV-APP/shared/UavMultiViewRecognition.hpp, 第 195 行。

API-01be92408d26 public

所属符号: `ndnsf::examples::uav::UavIncidentCoordinator::acceptMultiViewResult`

```
bool acceptMultiViewResult(const FusedRecognitionResult& result, uint64_t nowMs,
                           std::string* reason = nullptr);
```

源码: NDNSF-UAV-APP/ground-station/UavIncidentCoordinator.hpp, 第 66 行。

完整声明查询: [api/uav-reference.md](#)。

58 API 扩展与跨模块调用检查

AC-23 · Core 接口契约

58.1 新增服务的最小闭环

以 echo 服务为例：应用定义可序列化请求/响应，Provider 在 ACK 阶段只判断资格，Selection 通过后执行并返回；保存 scoped registration；User 用 AC-03 普通调用保存 requestId 并处理响应/超时；停服时关闭注册并排空工作。跨网络测试应覆盖无权限、无候选、错误响应与迟到回调，而不仅是本地 handler 直接调用。Core 不为这个服务增加领域字段。

58.2 新增 DI 策略的最小闭环

实现 AC-15 的 identity/enumerate 或 proposeRoles，用固定模型图和认证报价生成候选；交由共享 validator，准备/发布/再绑定后封存；将实现登记到实际使用的 registry/配置；验证默认调用链确实选中该策略。只增加类、只通过 mock 端口测试或只生成更多 API ID，都不能表示该功能接入产品。

58.3 设计维护的交付单元

API 变化同时登记输入/输出字段、单位、错误、状态、线程与资源、源码位置和验证证据。按 MANAGEMENT.md 更新当前设计；目标独立维护，PLANNED 不写成 existing。生成器验证声明，人工审阅验证解释能否指导调用；每个 Spec 在 spec-design-changes.md 记录前后行为，不用文件数量或模板段落宣称完整。

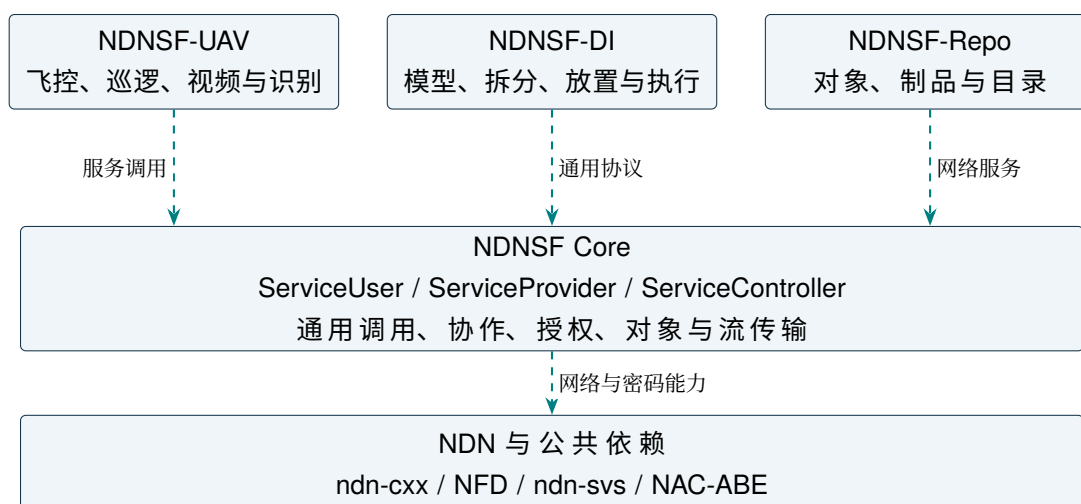
完整声明查询：api/core-reference.md。

59 图解：四模块关系、对象与关键流程

CURRENT · 箭头说明关系，正文与 API 契约说明完整行为

本章对照当前源码快照，图中省略非关键字段和 helper。实线带箭头表示明确标注的持有或流程，虚线表示调用、协议消息或目标步骤，不能仅凭连线推导继承、线程安全或远端成功。模块图不是部署拓扑；对象图不替代精确声明参考。各图源码入口见 [Design/diagrams/README.md](#)。图 G7 区分已接通的 PreparedModel 路径与保留的兼容 not-ready 分支。

59.1 G1：模块依赖与领域边界

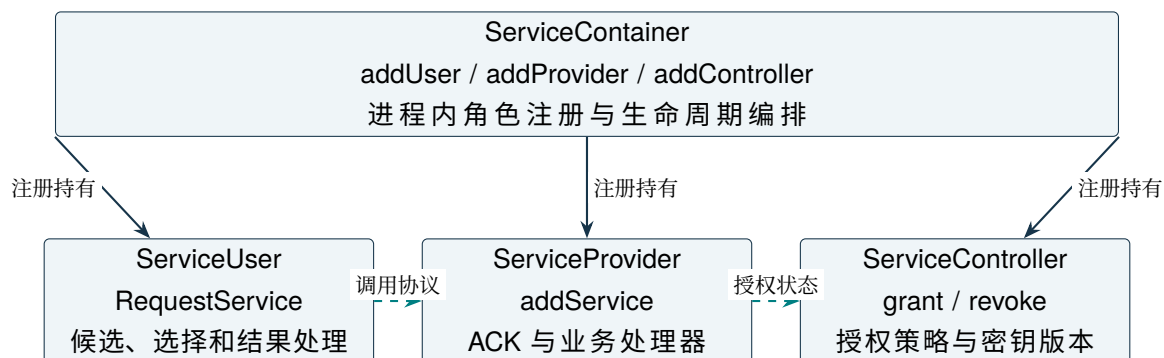


应用间关系：UAV 使用 Repo 保存录像；DI 使用 Repo 发布或读取制品。这两类依赖是应用编排，不要求 Core 理解视频帧、模型图或目录业务语义。UAV 的检测服务也不能直接等同于已接通完整原生 DI 请求链。

阅读范围 箭头只表示依赖方向，不表示每个组件都是独立进程，也不表示所有部署都必须启动 Repo。RepoCore 的本地存储操作不依赖一次 Core 网络调用。

正文入口：系统组成与职责边界；Repo 架构；DI 应用入口；UAV 服务容器。

59.2 G2: Core 角色组合与所有权

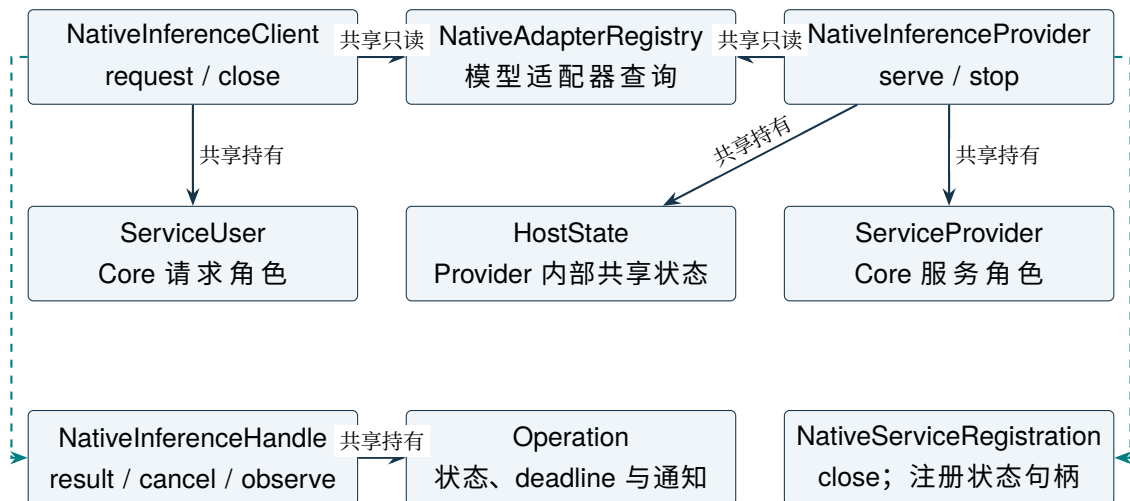


容器不是第四个 wire 角色，User/Provider/Controller 之间也不是继承关系。shared_ptr 重载共享持有；unique_ptr 重载转移到共享持有；引用重载使用不删除对象的包装，外部对象寿命须覆盖使用期。图中的“注册持有”不能统一解释成容器独占所有权。

生命周期 调用者、处理器及底层 Face 的实际回调线程由各 API 契约决定。容器停止、一次 invocation 取消、权限撤回是不同动作，不能由对象连线推导其等价性。

源码：ndn-service-framework/ServiceContainer.hpp、ServiceUser.hpp、ServiceProvider.hpp、ServiceController.hpp。详见 AC-01、Core 启动与角色章节。

59.3 G3: DI 原生入口与共享对象

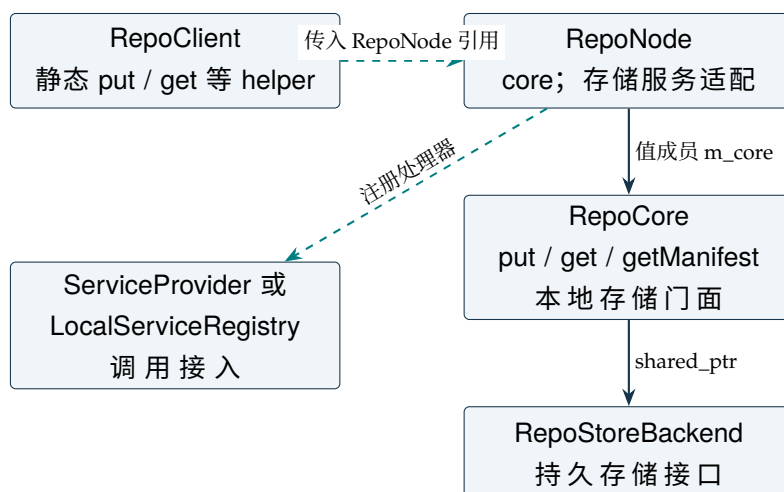


两侧虚线分别表示 request 返回 handle、serve 返回 registration。这是关键对象关系的选择性视图：Client 还持有准备、admission、grant、conversation 等协作对象，未全部展开。Registration 持有自己的 State；不把它画成 HostState 本身，也不把注册句柄关闭等同于已经停止远端任务。

接线边界 构造函数接收一个依赖，并不证明默认 request 路径已调用它。当前路径与目标调用阶段分别见各自 G7；Provider 局部能力不代表 requester 全链完成。

源码：NDNSF-DistributedInference/cpp/ndnsf-di/NativeInferenceClient.hpp、NativeInferenceClient.cpp、NativeInferenceProvider.hpp。详见 AC-14 与 AC-15。

59.4 G4: Repo 本地存储与服务适配

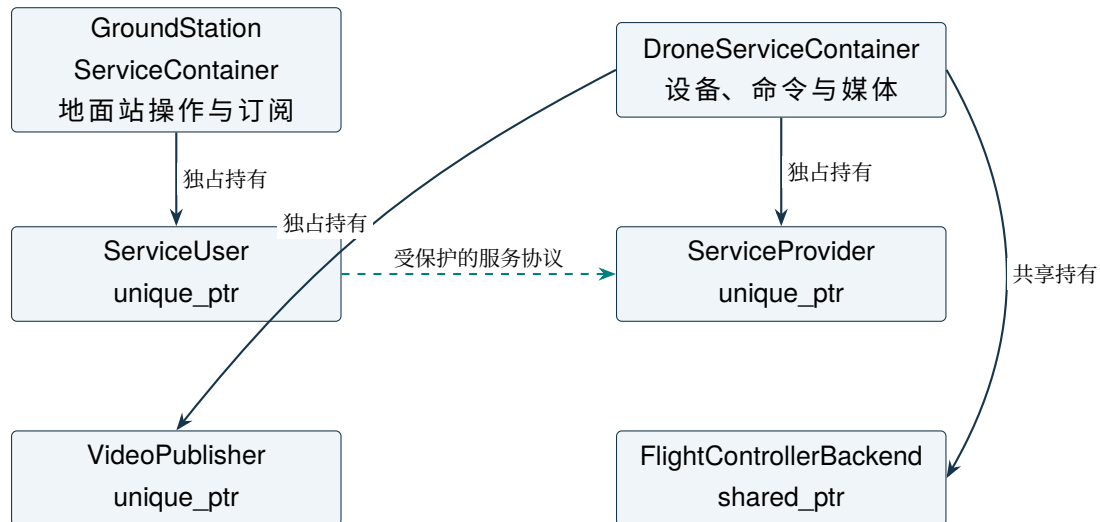


`RepoClient` 的直接 helper 不是拥有一个 `RepoNode` 的长期会话对象。`RepoNode` 的值成员 `RepoCore` 随节点销毁；底层 store 共享持有，其最后一个持有者决定释放。服务注册路径和本地 helper 是不同入口，不强制每次本地 `put` 都经过 `NDN`。

不要混为一层 大制品的 `PayloadStore/MetadataStore` 与普通 `RepoStoreBackend` 是不同职责，不在本图中伪造继承关系。Python 制品会话、目录和副本编排也不能由 C++ helper 类名推断为同一套完整网络实现。

源码：NDNSF-DistributedRepo/include/ndnsf-distributed-repo/RepoClient.hpp、RepoNode.hpp、RepoCore.hpp。对应 Repo 架构、持久化与 AC-10-AC-13。

59.5 G5: UAV 容器与设备适配

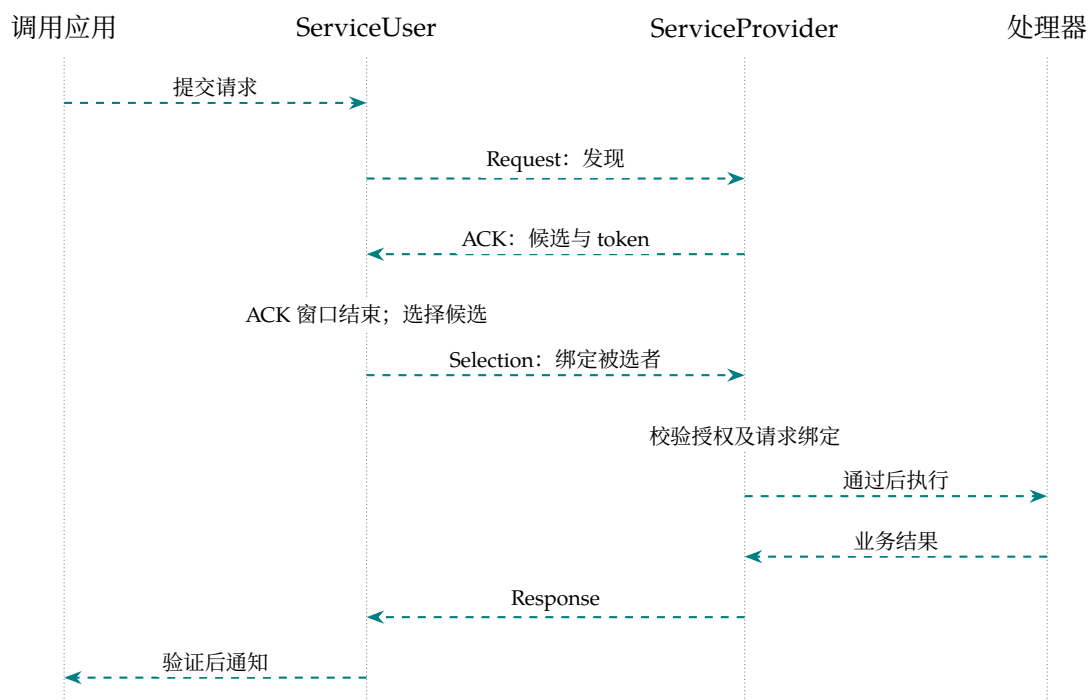


图中为两侧成员子集：Drone 也有 ServiceUser，地面站也有检测 Provider；省略不等于不存在。backend 抽象飞控访问，具体设备实现与 mock/SITL 由部署配置选择，不是 ServiceProvider 的子类。

释放与真实设备 容器对象释放、媒体流停止、mission 取消、飞控完成动作分别有自己的状态和清理契约。unique_ptr 只能说明本地对象所有权，不能证明外部设备已停止。

源码：NDNSF-UAV-APP/drone/DroneServiceContainer.inc.hpp、ground-station/GroundStationServiceContainer.inc.hpp。对应 UAV 容器、飞控与 AC-19。

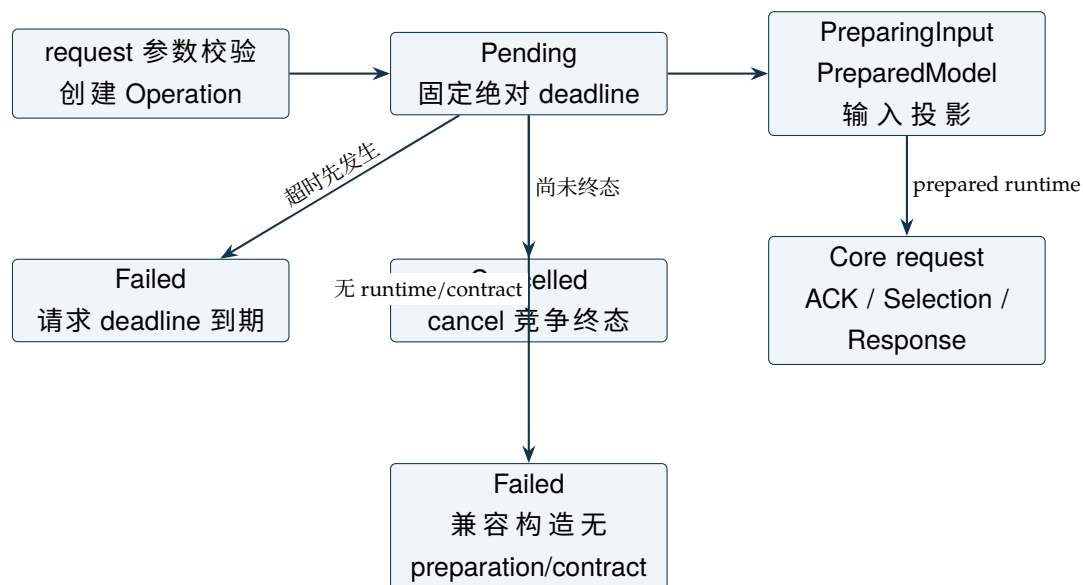
59.6 G6: Core 普通调用的主要时序



竖线是逻辑参与者，不是线程；横线概括协议/回调，不逐个展开 Interest、Data、签名与密钥获取。图只画成功主路径：无候选、授权失败、选择过期和执行错误须在对应阶段结束或拒绝，不能跳到成功回调。Controller 提供授权权威，不假定每次请求同步经过 Controller。

范围 Targeted 调用有单独的受保护路径，不能把本图删除 ACK/Selection 后当成其完整契约。Core 调用完成也不自动证明 UAV mission、Repo 副本或 DI 会话已完成。

59.7 G7: DI 原生请求当前可达状态

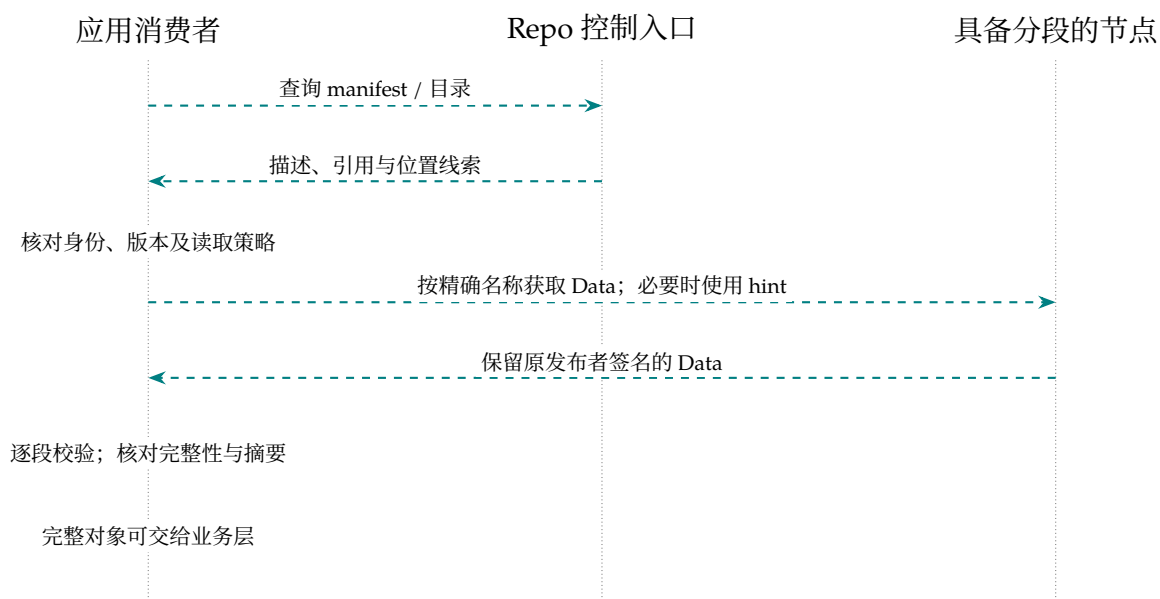


图混合展示内部 phase (PreparingInput) 与对外 status (Pending/Failed/Cancelled)，它们不是同一个枚举。PreparedModel 路径从输入投影进入 Core request，并由 C++ 过程矩阵验证 ACK/Selection/Response；没有 preparation/request contract 的兼容构造保留 NATIVE_REQUEST_PIPELINE_NOT_READY。

终态与等待 cancel、deadline 和 dispatch 失败竞争一次终态写入；已有终态不被后续路径覆盖。result(waitTimeout) 的局部超时只结束这次等待，不自动取消 operation。构造或提交前的非法参数异常未画成异步终态。

源码：PreparedModel.cpp 与 NativeInferenceClient.cpp 的 request、dispatchOperation、markTerminal、cancelOperation 和 result。声明中存在 Succeeded 等状态不代表默认路径已可达。Python facade 只转换参数、异常、GIL 与 asyncio，不另建请求状态机。

59.8 G8: Repo 控制面与数据面的读取顺序

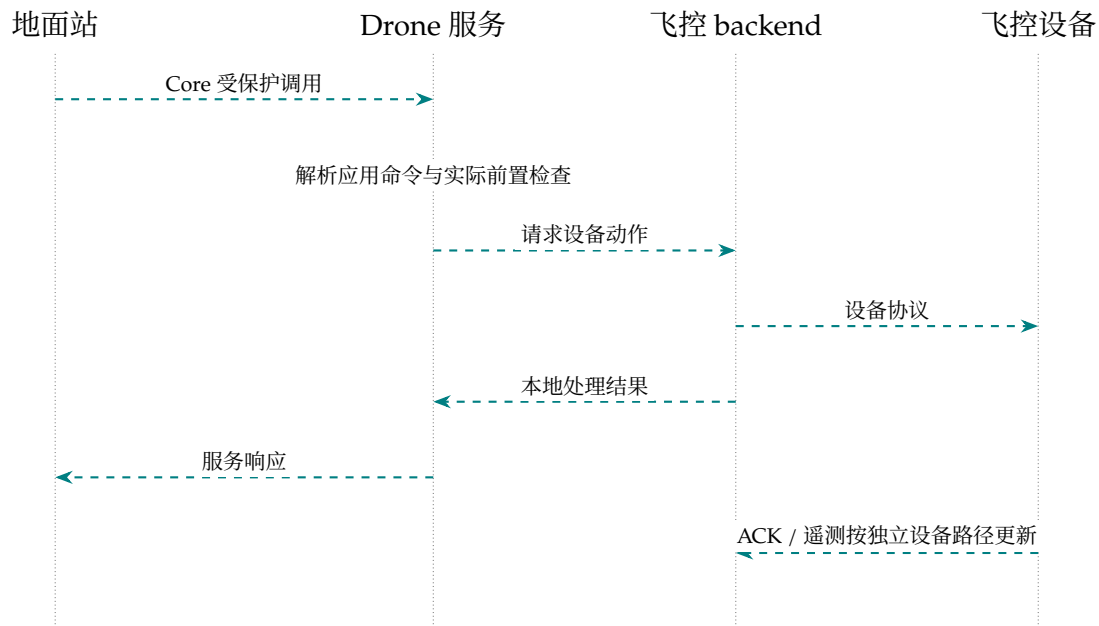


这是职责级时序，网络 helper/Python 制品会话的具体入口各有支持范围，不承诺所有 C++ RepoClient 方法都实现图中全部步骤。目录回复只是元数据，并不证明全部分段可达；来源节点可能与控制节点相同，也可能不同。

失败与恢复 缺段、错误签名、身份/摘要不匹配必须保留失败边界。恢复时沿既定操作身份核对已有内容；不能仅凭本地文件存在宣称下载完成。消费者负责业务解码，Repo 不解释张量或视频帧。

正文：Repo 大制品传输、目录与副本；源码入口见 R2、R7 及 AC-10-AC-13。

59.9 G9: UAV 命令响应与设备结果分层



此图强调两个结果通道，不承诺设备 ACK 总在服务响应之后到达，也不表示所有 backend 都等待同一种 ACK。真实返回/等待语义依命令和 backend 契约；mock 与真实设备必须分开解释。图中的“实际前置检查”不把 operatorLease 等字段自动当成已强制执行的授权。

任务层 一次服务响应不等于飞行完成；mission 含多个 invocation，补偿只处理缺失 part，并检查 task/attempt/assignment 绑定。取消本地操作不等同于无人机立即停止。真实飞行、断连恢复和 fail-safe 仍需对应实验验证。

正文：UAV 飞控、巡逻与任务补偿；源码：DroneServiceContainer、FlightControllerBackend 及 GroundStationServiceContainer，详细契约 AC-19、AC-20。